Module - 1

Introduction to Data Structures

Nithin Kumar
Asst professor
Dept of CS&E
VVCE, Mysuru

A Data Structure is a data organization, Management & Storage format that Enables Efficient Access & Modification. A Data Struct -ure is a Collection of data Values, the relationship among them & the functions/operations that can be applied to the data.

* Types of Data Structures :- There are two classes of Data Structures Such as
  * primitive Data Structures
  * Non-primitive Data Structures.

* primitive Data Structures :- are the Basic data Structures that directly operate upon the Machine Instructions.
→ primitive Data Structures are fundamental data types that are Supported by a programming language.
They have different representations on different Computers.
→ "Integers", "floating point nos", "character Constants", "String" & "pointers" come under this category.

* Non-primitive Data Structures :- are More Complicated data Structures & derived from primitive data Structures.
→ They Emphasize on Grouping Same or different data items with relationship between Each data item.
→ Non-primitive data Structures are also called "Compound data Structures".

Nithin VCE

Scanned by CamScanner

→ "Arrays", "Lists" & "files" come under this category.

→ Non-primitive data structures are classified as

* Linear data structures
* Non-Linear data structures.

* Linear data structures → Data structure where its values or elements are stored in sequential or linear order is called "Linear data structure".

Example → Arrays, Stacks, Queues, Linked Lists.

* Non-Linear data structures → Data structures where its values or elements are not stored in sequential or linear order is called "Non-Linear data structures".

Example → Trees & Graphs.

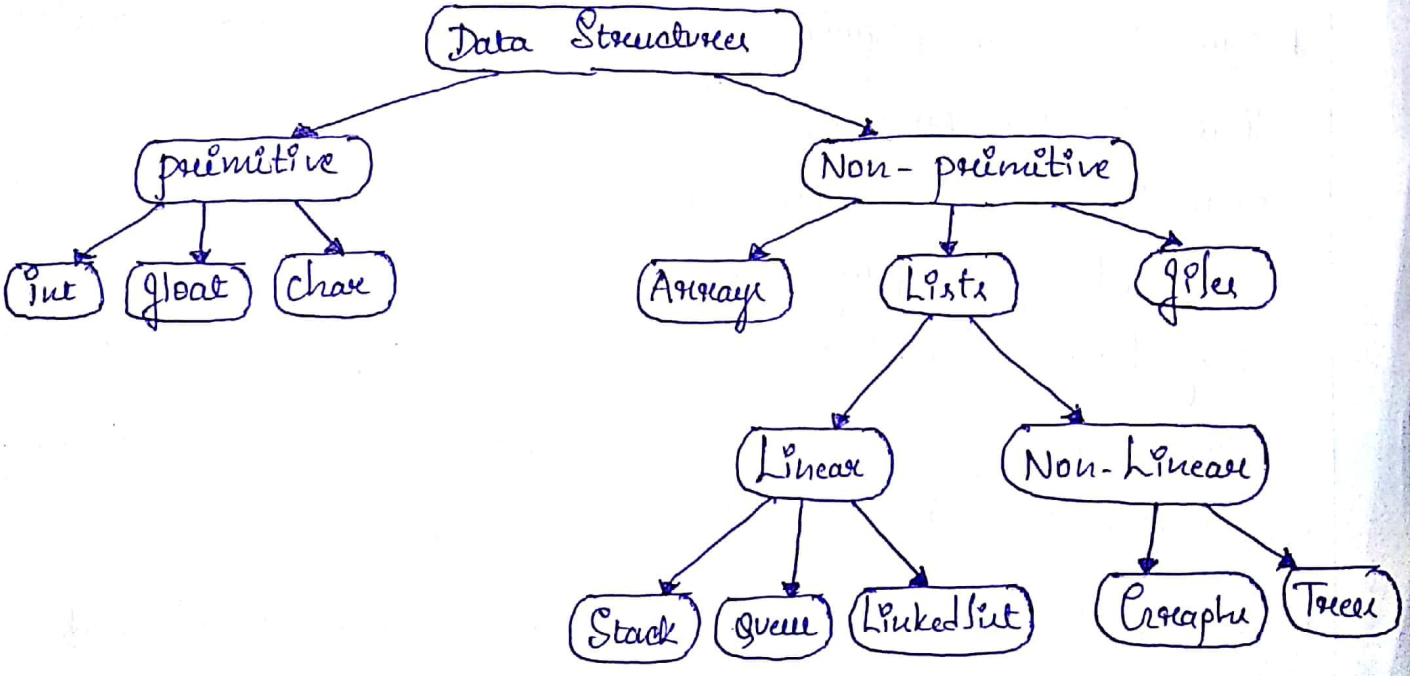


fig:- Classification of Data Structures

* Data Structure operations :-

The Basic operations that are performed on data structure are as follows

Nithin VCE

Scanned by CamScanner

* Insertion :- Insertion Means addition of a new data Element in a data Structure.

* Deletion :- Deletion means Removal of a data Element from a data Structure if it is found.

* Searching :- Searching Involves Searching for the Specified data Element in a data Structure.

* Traversal :- Traversal of a data Structure means processing all the data Elements present in it.

* Sorting :- Arranging data Elements of a data Structure in a Specified order is called Sorting.

* Merging :- Combining Elements of two Similar data Structures to form a new data Structure of the Same type, is called Merging.

* Algorithm :-

An Algorithm is a finite set of Instructions or logic, Written in order, to Accomplish a Certain predefined task.

→ Algorithm is not the Complete Code or program, it is just the Core logic (Solution) of a problem, which can be Expressed Either as an Informal high level description as "pseudo-code".

An Algorithm is said to be Efficient & fast, if it takes less time to Execute & Consumes less Memory Space.

Nithin, VCE

Scanned by CamScanner

→ The performance of an algorithm is Measured on the basis of following Metrics

* Time Complexity
* Space Complexity

* Time Complexity :- It is a way to represent the amount of time needed by the program to run to Completion.

→ Generally the time Complexity will be represented using "Asymptotic Notations" Such as

* Ω - Notation (Best - Case)
* Θ - Notation (Average - Case)
* O - Notation (Worst - Case)

* Space Complexity :- It is the amount of Memory Space required by the Algorithm, during the Course of it's Execution. Space Complexity Must be taken Seriously for Multi-User Systems & in Situations where limited Memory is Available.

→ An Algorithm generally requires Space for all Components.

* Instruction - Space → to Store Executable Version of prgm.
* Data - Space → to Store all Constants & Variables.
* Environment - Space → to Store Environment Info.

* Characteristics of an Algorithm :- Not all procedures can be Called an Algorithm. An Algorithm Should have the below Mentioned characteristics.

* Unambiguous → Algorithm Should be Clear & Unambiguous. Each of it's Steps (phase) & their Input/Output Should be Clear & Must lead to only one Meaning.

Nithin.WCE

Scanned by CamScanner

* Input → An Algorithm should have 0 or more well defined Inputs.
* output → An Algorithm should have 1 or more well defined outputs, & should match the desired output.
* Finiteness → Algorithm must terminate after a finite no of steps.
* Feasibility → Should be feasible with the Available resources.
* Independent → An Algorithm should have Step-by-Step directions which should be independent of any programming code.

* Review of Arrays :-

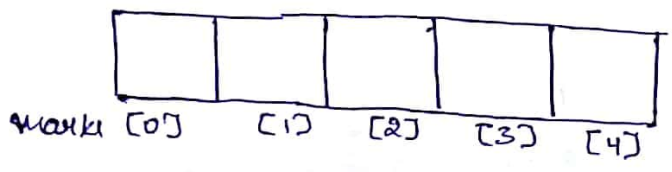
A Linear Array is a list of a finite number 'n' Homogenous data Elements, such that

* The Elements of the Array are referenced respectively by an Index set consisting of 'n' Consecutive nos.
* The Elements of the Array are Stored respectively in Successive Memory locations.

The number 'n' of Elements is called "length" or "Size" of the Array.

→ For declaring Arrays, we must provide "name", "data type" & "Size".

* Syntax :- Type Arr_name [Size];
* Example :- int marks [5];

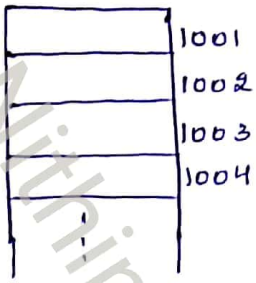


5 Memory locations with 2 byte Each.

Nithin VCE

Scanned by CamScanner

* Representation of Linear Arrays in Memory :- The Memory of the computer is simply a sequence of addressed locations as shown below



→ let A be a Array in the Memory

Loc (A[K]) = Address of the Element A[K] of the Array A.

→ Elements of A are stored in Successive Memory cells, the address of the first Element of 'A' is denoted by

Base (A)

Called Base Address of A.

→ Using the Above Mentioned, we can Calculate the Address of any Element of A by the formula

$$Loc(A[K]) = Base(A) + w(K - lowerbound)$$

where "w" is no of words per Memory cell for Array A.

* Array operations :- Different operations that can be performed on Arrays are

* Traversing
* Inserting
* Deleting
* Searching
* Sorting

Nithin VCE

Scanned by CamScanner

* Traversing :- In Traversing operation of an Array, Each Element of an Array is Accessed Exactly for once for processing. This is also called Visiting of an Array.

→ Each Element is Accessed in Linear order Either from left-to-right or from right-to-left

→ We can Easily Read, Write or process the Array Items using Appropriate programming Constructs Such as "for-loop", "while-loop", "do-while" loop Etc.

→ Consider the foll Example.

* Firstly, we need to declare the Array

```
int marks[6];
```

* If, we want to Read data Items from keyboard

```
for (i=0; i<n-1; i++)
{
    Scanf ("%d", &marks[i]);
}
```

* If, we want to display data Items Stored in Array

```
for (i=0; i<n-1; i++)
{
    printf ("%d", &marks[i]);
}
```

* Inserting :- Inserting new Element does not Mean Increasing its Size. It is required to Insert an Item at the Specified position.

→ The Steps to be followed for Insertion are

* Input new Element & position to Insert in Array
* Check for Invalid position
* Shift Items from the given Insertion position towards

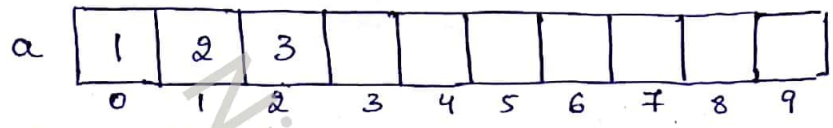
Nithin, VCE

Scanned by CamScanner

-right by one position.

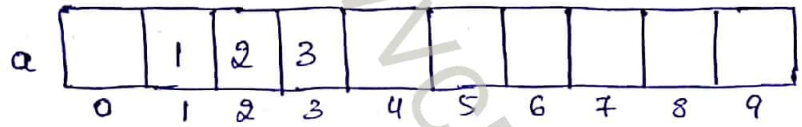
* Finally, Insert new item in vacant position.

→ Consider the foll Example of Array a[10] having three items in it Initially a[0] = 1, a[1] = 2 & a[2] = 3

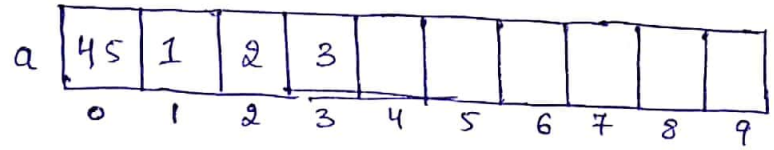


now, we need to Insert a no 45 at location a[0] = 45

→ Firstly, we have to Move Elements one Step towards right Such that a[1] = 1, a[2] = 2, a[3] = 3 & a[0] = Vacant



→ Then, we Insert a[0] = 45



→ Function to Insert an item at Specified position in Array

```
Int Insert (Int Item, Int a[], Int n, Int pos)
{ Int i;
  If (pos > n || pos < 0)
    { printf ("Invalid position");
      Return n;
    }
  for (i = n-1; i >= pos; i--)
    { a[i+1] = a[i]
    }
  a[pos] = Item;
  Return n+1;
}
```

Nishin VCE

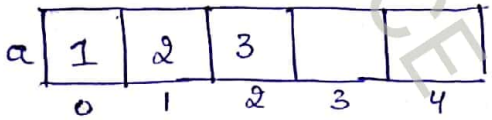
Scanned by CamScanner

* Deleting :- Is a process of deleting an Item from an unsorted Array based on position.

→ The Steps to be followed for deletion are

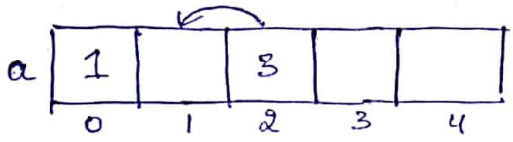
* Input the position of Item to be deleted from Array.
* check, whether the Given position is valid or not.
* Remove the Item from the Array.
* Finally, Move all Items from deleted position towards left by one position & update the no of Items.

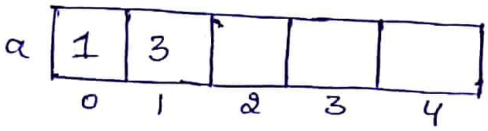
→ Consider the foll Example of Array a[5] having three Items in it Initially a[0] = 1, a[1] = 2, a[2] = 3



Now, we need to delete Item "2" at position a[1] = 2

→ Finally, we have to Move all Items from a[1] = 2 onwards towards left by one position.





→ Function to delete an Item from Specified position in Array

```
Put delete (Put a[], Put n, Put pos)
{
    Put i;
    If (pos > n || pos < 0)
        {
            printf ("Invalid position");
            Return;
        }
    printf ("Item deleted = %d\n", a[pos]);
    for (i = pos+1; i < n; i++)
        {
            a[i-1] = a[i];
        }
```

Nithin, VCE

Scanned by CamScanner

```
    return n-1;
}
```

* **Searching** :- is used to find whether a given no is present or not in an Array & if it is present then at what location it occurs.

→ The two important Searching techniques are

* Linear Search
* Binary Search

* **Linear Search** :- is a Simple Searching Algorithm is also called as Sequential Search
  * Start from the leftmost Element of Array & one-by-one Compare 'x' with Each Element of Array.
  * If 'x' Matches with an Element, Return the Index, otherwise Return -1.

→ Consider the foll Example.

arr[] =

10	20	80	30	60	50	110	100	130	170
0	1	2	3	4	5	6	7	8	9

x = 110

output = 6 i.e Element x is present at Index "6".

→ The function to Implement Linear Search is

```
int Linear (int key, int a[], int n)
{
    int i;
    for (i=0; i<n; i++)
    {
        if (key == a[i])
            return i;
    }
    return -1;
}
```

Nithin VGE

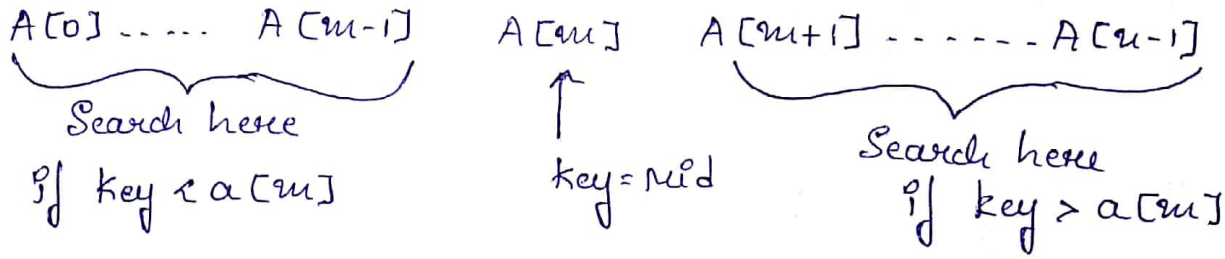
Scanned by CamScanner

* Advantages of Linear Search
  - → Very Simple
  - → Works well in Small Array
  - → Used to Search when the Elements are not Sorted.

* Disadvantages of Linear Search
  - → less Efficient (More no of comparisons are required).

* Binary Search :- Very Efficient Searching Algorithm on a Sorted list. The key Element is Searched with the Middle Element of the Array.
  - * If they are Equal, the position of Middle Element is returned.

  Else
  - * If the key is found to be greater than Middle Element, then the Searching is Applied on Second half of Array. Otherwise on the first half of Array.



→ The function to Implement Binary Search is

```
int bin_Search (int key, int a[], int n)
{
    int low, high, mid;
    low = 0;
    high = n-1;
    while (low <= high)
    {
        mid = (low + high)/2;
```

Nithin, VCE

Scanned by CamScanner

```
    if (key = a[mid]) return mid;
    if (key < a[mid]) high = mid-1;
    if (key > a[mid]) low = mid+1;
  }
  return -1;
}
```

Nithin VCE

* Advantages of Binary Search
  - → Simple Searching technique
  - → Very Efficient than Linear Search (Requires lesser no of Comparisons)
* Disadvantages of Binary Search
  - → Applicable only on the Sorted list of Elements.

* Sorting :- Rearranging the Elements in Linear Array in Ascending / Descending order.

→ There are different Sorting techniques Such as Bubble sort, Selection sort, Merge sort, Quick sort Etc.

→ The Sorting techniques which make use of Brute-force Approach (Straight-forward Approach) are
  - * Selection Sort
  - * Bubble Sort

* Selection Sort :- Algorithm Sorts an array by repeatedly finding the Minimum Element (considering Ascending order) from unsorted part & putting it at the beginning.

→ The Algorithm maintains two Sub arrays
  - * The Subarray which is Already Sorted.

Scanned by CamScanner

* Remaining Subarray which is unsorted

In Every Iteration of Selection Sort, the Min Element from the unsorted Subarray is picked & Moved to the Sorted Array.

→ Consider the foll Example of Selection Sort

arr[] =

64	25	12	22	11

* finding the Min Element in arr[0...4] & place it in beginning

11	25	12	22	64

arr[1..4]

* finding the Min Element in arr[1...4] & place it in beginning

11	12	25	22	64

arr[2..4]

* finding the Min Element in arr[2..4] & place it in beginning

11	12	22	25	64

arr[3..4]

* find the Min Element in arr[3..4] & place it in beginning & Remaining Element will be the last one in list.

11	12	22	25	64

→ The function to Implement Selection Sort in C

```
Void Selsort (int a[], int length)
{
    int indexMin;
    for (int s=0; s<length-1; s++){
        indexMin = s;
        for (int k= s+1; k<length; k++){
            if (a[k] < a[indexMin])
                indexMin =k;
```

Nithin VDE

Scanned by CamScanner

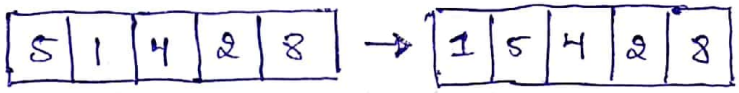
```
    }
      Swap (a[s], a[indexMin]);
  }
}
```

* Advantages of Selection Sort
  - → Easy / Simple Implementation
  - → Useful when no of items are less
  - → Can be used when Memory is less

* Disadvantages of Selection Sort
  - → It is slower when no of Elements are More.
  - → performance varies with initial order of Input.

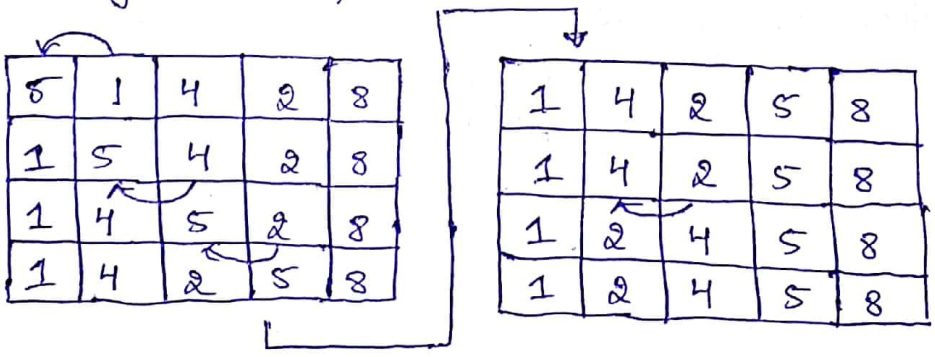
* Bubble Sort :- Bubble Sort is the Simplest Sorting Algorithm that works by repeatedly Swapping the adjacent Elements, if they are in wrong order.

→ For Example



Here, Algorithm Compares the first two Elements & Swaps. Since 5 > 1.

→ Consider the full Example



→ The function to Implement Bubble Sort

```
Void bubble_Sort (int a[], int n)
{
    int i;
    int j;
    int temp;
```

Nithin, VCE

Scanned by CamScanner

```
for (j=1; j<n; j++)
    {
        for (i=0; i<n-j; i++)
            {
                if (a[i] >= a[i+1])
                    {
                        temp = a[i];
                        a[i] = a[i+1];
                        a[i+1] = temp;
                    }
            }
    }
```

Nithin VCE

* Advantages of Bubble Sort
  - → Simple & Easy to Implement
  - → Straight forward Approach for Sorting.

* Disadvantages of Bubble Sort
  - → More no of Comparisons are required (Slow).

* Structures :-

Structures are User-defined data-types & using Structure we can define a data-type which holds More than one Element of different types.

→ Like primitive data types Such as Int, float, char Etc, using Structure we can develop User-Datatypes.

→ The "General Syntax" of Structure

```
Struct <Identity>
    {
        data-type  Ele1;
        data-type  Ele2;
            :
    };
```

Where <Identity> = name of Structure, Ele1, Ele2 = Data members.

Scanned by CamScanner

→ Consider the foll Example of Employee Structure

```
Struct Employee
{
    int eno;
    char ename[20];
    float esal;
};
```

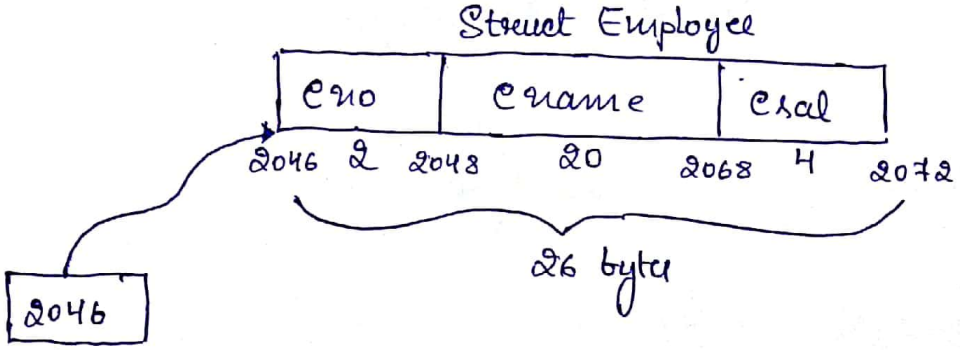
In Array only Homogenous Elements are Stored so using Arrays we cannot process info like Employee data, Student data Etc. so we'll make use of Structures.

→ Just by declaring the Structure, the Memory will not be allocated, Because Structure is just a "data-type". we need to declare Variable in Main function for Memory allocation. As shown below

```
Void main()
{
    int a;
    Struct Employee e;
}
```

→ Based on different data members present inside the Structure the no of bytes of Memory will be allocated.

→ In the above Example "Struct Employee", we have Integer = 2 bytes, character(20) = 20 bytes, float no = 4bytes. totally the Memory Allocation will be as shown



Variable holds the Initial Address.

Nithin, VCE

Scanned by CamScanner

→ Once the Memory is allocated, we can Store / Retrieve the Info in Structure using (.) operator also called as "Access". Such as c.eno, c.ename, c.esal to access the members of Structure. using

* Member operator (.)
* Structure pointer operator (->)

* Type-defined Structure :- Type-def is a powerful tool that allows programmer to define & then use their own data-types

→ Increase the readability & Decrease some considerable devel-opment time. So, developers Generally use "Typedef" to name the Structure as a whole.

→ The "General Syntax" to declare Type-defined Structure

```
Typedef Struct
{
    Type1 member1;
    Type2 member2;
    :
} Type_ID;
```

→ Consider the foll Example of Type-defined Structure

```
Typedef Struct
{
    Char name[10];
    Int roll_no;
    Float Avg_Marks;
} STUDENT;
```

Since "STUDENT" is the type Created by the User, using the keyword "typedef". It can be called as "User-defined Datatype" So we can use "STUDENT" as Datatype & Declare the Variables.

```
STUDENT CSE, ISE;
```

Nithin VCE

Scanned by CamScanner

* Structures within Structures (Nested Structures) :- Structures can be nested within other Structures as shown below

```
Struct Complex
  {
    int image_value;
    float real_value;
  };
Struct number
  {
    Struct Complex comp;
    int real;
  } num1, num2;
```

* passing Structure to Function :- It can be done in 3 ways
  * passing Structure to function by value
  * passing Structure to function by address
  * Declare

* passing Structure to function by value, It means the whole structure is passed to another function with all members & their values.

  Ex:- void func (Struct Student record)

* passing Structure to function by reference, It means only the address of the Structure is passed to Another function.

  Ex:- void func (Struct Student *record)

Finally Structure Variables also can be declared as global variables as we declare other variables in C. then it is visible to all the functions in a program, in this scenario we dont need to pass the Structure to Any function Separately.

Nithin VCE

Scanned by CamScanner

* Advantages of Structure
  - → Structures in C are used to combine more than one data
  - → Structures give you flexibility to access data easily
  - → Increase the readability of program.

* Union :- Unions are declared, created & used exactly the same as Structures, Except the foll one key difference
  - * Structs allocate Enough Space to Store all of the fields in the Struct. The first one is stored at the beginning of Struct, the second is stored after that & so on.
  - * Unions only Allocate Enough Space to Store the largest field listed & all fields are Stored at the same Space.

→ The "General Syntax" used to declare union

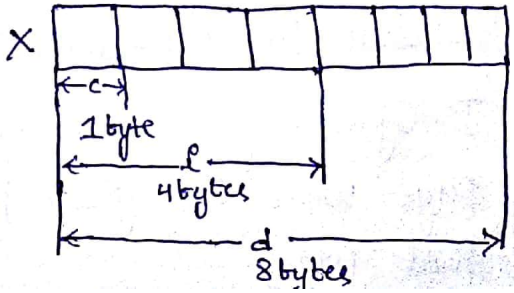
```
union <tag_name>
{
    Type1  member1;
    Type2  member2;
    :
};
```

→ Consider the foll Example of union

```
Typedef union
{
    int i;
    double d;
    char c;
} item;
```

→ Once we declare the union, this is how Memory allocation take place

item x; ⟶



Nithin VICE

Scanned by CamScanner

By looking at the above declaration, the compiler will reserve the Memory whose Size is that of largest member. Since the "double" is the largest datatype member of union in above Ex Sizeof (double) = 8 byte of Memory is reserved for Variable 'X'.

*Difference between Structure & union

	Structure	Union
*Keyword	* The keyword "Struct" is used to define a Structure	* The keyword "union" is used to define a union.
*Size	* Greater than or Equal to the Sum of Sizes of its members	* Size of union is Equal to the Size of largest member
*Memory	* Each member in Structure is assigned unique Storage area	* Memory allocated is Shared by Individual Members of union.
* Value Altering	* Altering the value of one member will not affect others.	* Altering the value of one member will alter others member values.
*Initialization of members	* Several members of a Structure can Initialize at once.	* Only the first member of a union can be Initialized.

* pointers :- Is a programming language object that Stores the Memory address of Another Value located in computer Memory.

→ A pointer references a location in memory & obtaining the Value Stored at that location is known as "Dereferencing" the pointer.

→ For Ex, A page no in a books Index Could be considered a pointer to Corresponding page.
Dereferencing Such a pointer would be done by flipping to the page with the given page no & Reading the text found on that page.

Nithin VVCE

Scanned by CamScanner

→ Using pointers significantly improves performance for repetitive operations like traversing iterable data structures such as strings, lookup-tables, Tree structures Etc.

→ The "General Syntax" for declaring pointers

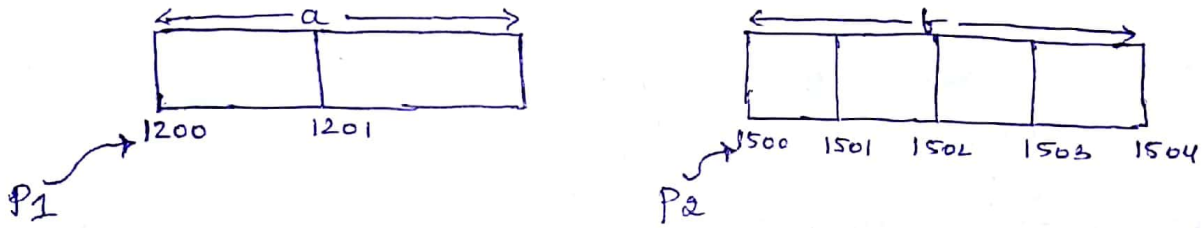
Datatype *ptr;

where ptr = pointer_name

→ Consider the foll Example of pointers

```
int a=10;
int b=4.5;
int *p1;
int *p2;
p1 = &a;
p2 = &b;
```

where '&' is known as "Address operator", so "p1" holds the address of a, "p2" holds the address of b as shown



* pointers as function Arguments :- pointers can be passed as an Argument to a function & this Methodology is known as "Call-by-address/Reference".

→ Consider the foll Example for pointers as function Arguments

```
#include <stdio.h>
void fun (int *x)
{
    *x = 5;
}
void main ()
{
    int a;
    fun (&a);
```

Nithin, VVCE

Scanned by CamScanner

```
    printf("a = %d", a);
}
```

Initially "a" contains Garbage value, then a is being passed to function() which as parameter int *x

int *x = &a

Inside func(), the value of *x = 5, value at address x = 5, Hence the "a" gets value = 5.

* Array & pointers :- Array is a collection of Elements, to access all the Elements through pointers, we need to traverse the array Even for this, only Starting address is Sufficient.

The Starting address of an array is known as "Base Address"

→ The Base Address of Array can be Assigned to pointers as

```
int a[10];
int *p;
p = a;
```

or

```
p = &a[0]
```

Either the name of Array or the Address of first Element can be Assigned to pointers.

* Pointer Arithmetic :- The diff Arithmetic operations Applicable on pointers are

* Increment/Decrement → If pointer is p then p++ & p--

Ex:-
```
int *p;
float *ptr;
p++; ptr--;
```

* Addition / Subtraction → If "p" & "q" are pointers of Same data type

Ex: q = p-2; q = p+5; are valid.

* Multiplication/Division → with de-referenced Values is allowed

a = (*p1) * (*p2)

Nithin VCE

Scanned by CamScanner

Multiplication/Division of a pointer with Another pointer or with an Integer value is not allowed. Such as P1/P2, P1*P2, P1*3 Etc.

* Compare → Can be Compared with Another pointer using Relational operator

Ex:- P1>=P2 , P1!=P2 Etc.

* Function Returning pointers:- A function can normally return any basic datatype (Int, float, Etc) or user-defined datatypes like (Struct, union)

→ In Similar manner a function can return a derived data type like pointer.
The pointer/Address returned by function can be a pointer to any data type.

→ Consider the foll Example

```
#Include <Stdio.h>
 Int *func()                    // function return type is pointer
   { Int a=5, *p;
     p=&a;
     return p;
   }
 Void main()
   { Int *x;
     x=func();
     Printf("%d", *x);
   }
```

* Dynamic Memory Allocation:- Generally if you declare any primitive variable

```
Int a;
```

The Memory Allocation will be done at compile time it is called "Static Memory Allocation" (Memory fixed).

Nitin, VCE

Scanned by CamScanner

→ In case of Arrays also Size is fixed, this comes under Static Memory allocation. as well as Structure with fixed no of records Such as

Int arr[5]; Struct Emp e[5];

The Memory will be allocated in runtime only but why we call "Static" is the Memory is fixed.

→ The C programming language Manages Memory

* Statically
* Dynamically

→ Neither "Static" nor "Automatic" duration of Memory is adequate for all Situations.

Automatic - allocated data cannot persist across Multiple function calls, while Static data persists for the life of the program whether it is needed or not.

→ In Many Situations the programmer requires greater flexibility in managing the lifetime of allocated Memory

These limitations are avoided by using "Dynamic Memory - allocation" in which Memory is more Explicitely Managed.

→ The C dynamic Memory allocation functions are defined in "Stdlib.h"

* Malloc → Allocates the Specified no of bytes.
* Calloc → Allocates the Specified no of bytes & Initializes them to zero.
* Realloc → Increases/Decreases the Size of Specified block of Memory.
* Free → Releases the Specified block of Memory back to System.

* Malloc :- "Malloc()" allocates Memory block of given Size & returns a pointer to the beginning of the block.

Malloc() doesn't Initialize the allocated Memory.

Nitin VIGE

Scanned by CamScanner

→ The "General Syntax" of Malloc()

```
void * malloc (size_t size)
```

Malloc() takes a single Arguement (Amount of Memory to allocate in bytes)

* Calloc :- Calloc() allocates Memory block of given size & initialized the allocated memory block to zero. If we try to Access the content of these blocks then we'll get '0'.

→ The "General Syntax" of Calloc()

```
void * calloc (size_t num, size_t size);
```

Calloc() takes two arguments (No of blocks to be allocated & size of Each block).

→ After Successful Allocation in Malloc() & Calloc(). A pointer to the block of Memory is returned, otherwise Null value is returned which indicates the failure of allocation.

→ Consider the foll Example of dynamic Memory allocation for array of 5 integers using Malloc() & Calloc()

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int * arr = Malloc (5 * sizeof(int)); // Malloc()
    free(arr);
    int * arr = Calloc (5, sizeof(int)); //calloc()
    free(arr);
}
```

It would be better to use Malloc() over Calloc(), unless we want the zero initialization because Malloc() is faster than Calloc().

* Realloc :- Reallocates the given area of Memory. It must be previously allocated by Malloc() or Calloc() & Not yet freed with a call "free".

Nithin VCE

Scanned by CamScanner

→ The Reallocation is done by Either Expanding/reducing the Exis-ting area pointed by ptr.

→ The "General Syntax" for realloc() is

```
void *realloc (void *ptr, size_t new_size)
```

Where "ptr" → pointer to the memory area to be reallocated
"new_size" → new size of array in bytes.

→ Consider the foll Example of realloc()

```
int *pa = malloc (10 * sizeof(*pa)); // Allocation
int *pb = realloc (pa, 100 * sizeof(*pb)); // reallocation
```

* Free() :- Deallocates the memory previously allocated by a call to Calloc(), Malloc() or realloc().

→ The "General Syntax" for free()

```
void free (void *ptr)
```

Where "ptr" → pointer to a memory block previously allocated with Malloc(), Calloc() or realloc().

* Common Errors of Dynamic Memory Allocation

* Not checking for allocation failures → Memory allocation is not Guaranteed to Succed & May instead return a Null pointer. Using the returned value without checking, if Allocation is Successful invokes "undefined behaviour" leads to Crash.

* Memory leaks → failures to deallocate Memory using "free" leads to buildup of non-reusable Memory, which is no longer used by program. This wastes Memory resources & can lead to allocation failures.

* Logical Error → All allocations must follow the same pattern

Malloc() or Calloc() → realloc() → free

failures to adhere this pattern or Calling "free" twice etc

Nithin VGE

Scanned by CamScanner

Causes a "Segmentation fault" & results in crash of the program.

* polynomials :- polynomial object is a homogenous ordered list of pairs <Exponent, coefficient> where each co-efficient is unique.

→ Operations on polynomials includes returning the degree, Extr-acting co-efficient, addition, Multiplication Etc.

→ The different types of polynomial operations are

* Representation
* Addition
* Multiplication

→ A polynomial is an Expression that contains more than two terms. A term is made up of co-efficient & Exponent.

Ex:- $p(x) = 4x^3 + 6x^2 + 7x + 9$

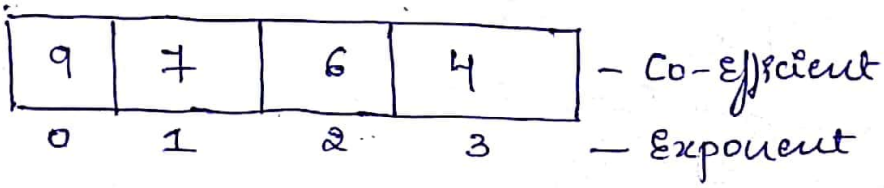
→ A polynomial can be represented using

* Array
* Linked List

→ Array representation assumes that the Exponents of the given Expression are arranged from 0 to highest value (degree) which is represented by the index of array begining with '0'. The co-efficients of the respective Exponent are placed at the Appropriate index in the Array.

→ Consider the foll Example of polynomial using Array Repr-esentation.

$$p(x) = 4x^3 + 6x^2 + 7x + 9$$



Within VCE

Scanned by CamScanner

→ A polynomial may also be represented using a linked list, A structure may be defined such that it contains two parts one is the co-efficient & second is corresponding Expo -nent.

```
Struct polynomial
  { int co-efficient;
    int Exponent;
    Struct polynomial *next;
  };
```

→ Consider the Example of polynomial represn using Linked List

$p(x) = 4x^3 + 6x^2 + 7x + 9$

→ Addition of two polynomials using Arrays is Straight-forward Method, Since both the Arrays may be added up Element wise begining from 0 to n-1, resulting in addition of two polynomials.

$p(x) = 4x^3 + 6x^2 + 7x + 9$ ⟶

9	7	6	4
0	1	2	3

+

$q(x) = 3x^2 + 2x + 4$ ⟶

4	2	3
0	1	2

$Sum = 4x^3 + 9x^2 + 9x + 13$ ⟶

13	9	9	4
0	1	2	3

→ Function to add two polynomials

```
int add_poly (POLY p1[], int m, POLY p2[], int n, POLY p3[])
{
    int i, k, cf1, px1, pos, sum;          // cf1 = co-efficient of poly 1
    k=0;                                   // px1 = Exponent of poly 1
    for (i=0; i<m; i++)
    {
```

Scanned by CamScanner

```
cf1 = p1[i].cf;                    // Access Each Item of poly 1
px1 = p1[i].px;
pos = Search (px1, p2, n);         // get position px1 in poly2
    if (pos > 0)                   // px1 found in poly2
    {
        Sum = cf1 + p2[pos].cf     // Add Co-Efficients
            p3[k].cf = Sum;        // Insert the Sum into poly3
    }
    Else
        p3[k].cf = cf1             // Insert the co-efficient of
                                   poly1 directly to result
                                   poly3.
        p3[k].px = px13
        k++;
    }
    return k;
}
```

* **Self-referential Structures:-** A Self referential Structures is used to create data Structures like linked list, Stack etc

→ A Self referential Structure is one of the data Structures which refer to the pointer to point another Structure of the same type.

→ For ex:- A linked list is supposed to be a Self-referential data Structure. The next node of a node is being pointed, which is of the same Struct type.

→ The "General Syntax" of Self-referential Structure is

```
Struct Struct_name
{
    Datatype name;
    Struct_name * pointer_name;
};
```

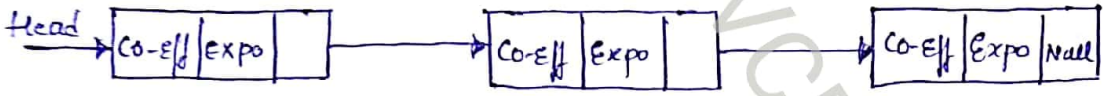
Nithin, VCE

Scanned by CamScanner

→ Consider the foll Example of polynomial representation using linked list

```
Struct polynomial
{
    int Co-efficient;
    int Exponent;
    Struct polynomial *next;
};
```

→ The Above Syntactical representation of polynomial can be pictorically represented as



* Sparse Matrix :- A Matrix can be defined with 2-D Array Any Array with m → cols & n → rows represents a m x n Matrix.
There May be a Situation in which a Matrix Contains More no of zero values than non-zero Values Such Matrix is known as "Sparse Matrix".

→ Consider the foll Example of Sparse Matrix

$$\begin{bmatrix} 10 & 0 & 0 & 40 \\ 11 & 0 & 22 & 0 \\ 0 & 0 & 0 & 0 \\ 20 & 0 & 0 & 50 \\ 0 & 15 & 0 & 25 \end{bmatrix}$$

→ A Sparse Matrix Can be represented using two Repsn
* Triplet Representation
* Linked list Representation

* Triplet representation :- we Consider only non-zero values along with their row & Column Index values.

Scanned by CamScanner

→ In this repsn, the 0th row stores "total no of rows", "total no of cols" & "total no of non-zero values" in Matrix

	0	1	2	3	4	5
0	0	0	0	0	9	0
1	0	8	0	0	0	0
2	4	0	0	2	0	0
3	0	0	0	0	0	5
4	0	0	2	0	0	0

⇒

Rows	Cols	Value
5	6	6
0	4	9
1	1	8
2	0	4
2	3	2
3	5	5
4	2	2

(Triplet repsn)

In above Example Matrix, there are only 6 non-zero Elements. The first row of triplet repsn is filled with values 5, 6 & 6 which indicates that it is a Sparse Matrix with 5-rows, 6-cols & 6-non-zero Values.

→ Second row is filled with 0, 4 & 9 which indicates the value in the Matrix at 0th row, 4th col is '9' & so on.

* Linked representation :- we use linked list data structure to represent a Sparse Matrix.

→ There are two diff nodes in representation

* Header Node
* Element Node

* Header Node → Consists of three fields such as "Index", "Value", "down & right".

* Element Node → Consists of five fields such as "row", "column", "Value", "down/up" & "right"

Index	Value
Down	Right

(Header Node)

row	col	value
Down/ up	Right	

(Element Node)

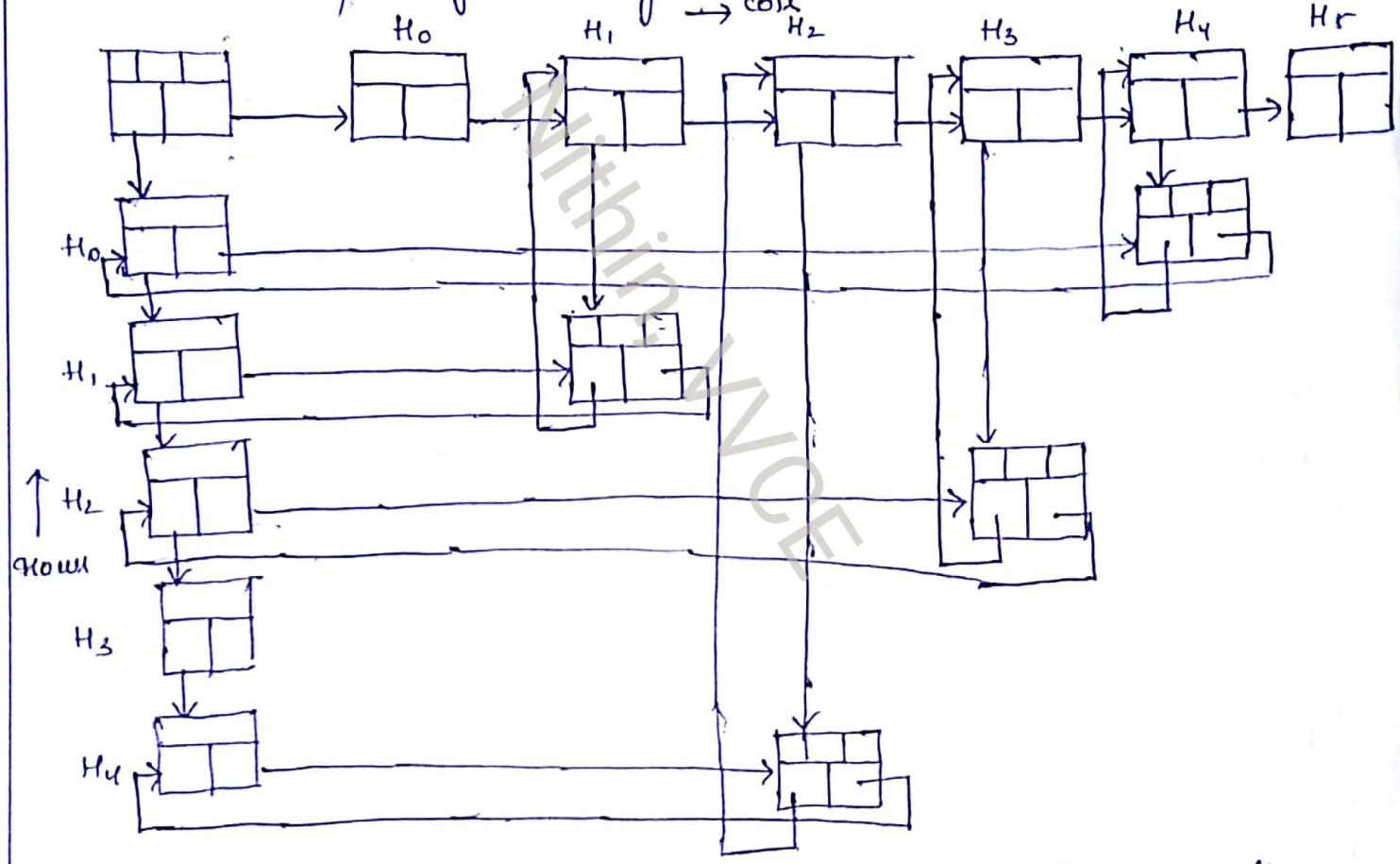
Within VCE

Scanned by CamScanner

→ Consider the Joll Example

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

The Linked Repsn of Above given Matrix is



In the above representation, the "Element" Node Indicates the non-zero value, the "Header" Node Indicate the row's & col's.

* Simple program accepts a Matrix of order mxn & Verifies whether it is a Sparse Matrix or not

```
#Include <stdio.h>
    Void main()
    {
      Static Int arr [10] [10];
       Int i, j, m, n;
       Int counter = 0;
       printf ("Enter the order of Matrix \n");
       Scanf ("%d %d", &m, &n);
```

Scanned by CamScanner

```
printf ("Enter the Co-efficients of Matrix\n");
for (i=0; i<m; i++)
    { for (j=0; j<n; j++)
        { scanf ("%d", &arr[i][j]);
            if (arr[i][j] == 0)
                { ++ counter;
                }
        }
    }
if (counter > ((m*n)/2))
    { printf ("the given Matrix is Sparse");
    }
  else
    printf ("the given Matrix is not Sparse");
}
```

* Transpose of a Matrix :- is obtained by changing row Elements into Column Elements & Column Elements into row Elements is called "Transpose of a Matrix"

→ The Algorithm to perform Transpose

```
Transpose (int b1[][3], int b2[][3])
{ int i, j, k, n;
  b2[0][0] = b2[0][1];
  b2[0][1] = b2[0][0];
  b2[0][2] = b2[0][2];
  k=1;
  for (i=0; i< b[0][1]; i++)
      for (j=1; j<=n; j++)
          if (b2[j][1] == i)
              { b2[k][0] = i;
                b2[k][1] = b2[j][0];
                b2[k][2] = b2[j][2];
                k++;
              }
}
}
```

Nithin VICE

Scanned by CamScanner

* Dynamically Allocated Arrays :- Allocate the Array during the run-time, we can call the function to allocate Memory

```
Int *Ptr n, i;
Malloc (Ptr, n* Sizeof (Int));
```

It will allocate 'n' Sized array to Ptr, "Malloc" is a Macro we can use to allocate memory, which is already Created before use.

* 2D Arrays :- C uses array-of-Arrays representation to represent a Multidimensional array.

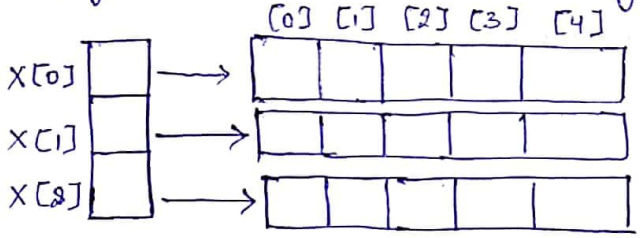
→ A 2D Array is represented as one-dimensional array in which Each Element is, itself a one-dimensional array.

→ To represent the two-dimensional array

```
Int x [3] [5];
```

We Create a 1-D Array x whose length is '3', Each Element of x is a 1D Array whose length is 5

→ The below figure Shows the Memory Structure



Arrays-of-Arrays representation

→ The following code creates a 2D Array at run time

```
Int** make2dArray (Int rows, Int cols)
    Int **x, i;
    Malloc (x, rows * Sizeof (*x));
    for (i=0; i<rows; i++)
        Malloc (x[i], cols * Sizeof (**x));
    Return x;
```


Nithin VNCE

Scanned by CamScanner

The above declared function can be used in foll ways

```
int **myArray;
myArray = make2dArray (5,10);
myArray [2] [4] = 6;
```

→ The Second line Allocates memory for a 5 by 10 2D-Array of Integers & third line assign the value 6 to the [2] [4] Element of the Array.

* Multi-Dimensional Array :- C programming language allows Multi-dimensional array.

→ The General form of Multidimensional Array declaration

```
Datatype name [size 1] [size 2] .... [size N];
```

→ Consider the foll Example to create a 3D Integer Array

```
int threedim [5] [10] [4];
```

→ The Simplest form of Multidimensional array is the two-dimensional array.
A two-dimensional Array is, in Essence, a list of one-dimensional Array. To declare a two-dimensional Integer Array of size [x] [y].

```
Datatype ArrayName [x] [y];
```

→ A two-dimensional Array a, which contains three rows & four columns can be shown as follows

	Col0	Col1	Col2	Col3
Row0	a[0] [0]	a[0] [1]	a[0] [2]	a[0] [3]
Row1	a[1] [0]	a[1] [1]	a[1] [2]	a[1] [3]
Row2	a[2] [0]	a[2] [1]	a[2] [2]	a[2] [3]

Thus, Every Element in the Array a is identified by an Element name of the form a[i] [j], where 'a' is the name of Array

Nithin, VCE

Scanned by CamScanner

and "i" & "j" are Subscripts that uniquely Identify Each Element in "a".

→ Multidimensional Arrays May be Initialized by Specifying bracketed Values for Each row. Following is an array with 3 rows & Each row has 4 Columns

```
int a[3][4] = {{0,1,2,3},
               {4,5,6,7},
               {8,9,10,11}
              };
```

→ The foll Initializations is Equivalent to previous Example

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

→ Accessing Two-dimensional array Elements by using the Subscripts i.e "row Index" & "col Index" of the Array

For Example :- `int val = a[2][3];`

* Strings :- Strings are defined as an array of characters. The difference b/w a Character array & a String is the String is terminated with a Special Character '\0'.

→ Declaring a String is as Simple as declaring a one dimensional Array

```
char Str_name[size];
```

→ The following declaration & Initialization Create a String Consisting of word "NITHIN" to hold the null Character at the End.

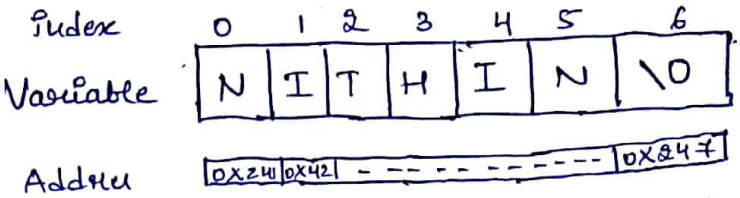
The Size of the character array Containing the String is one More than the no of Characters in the word "NITHIN".

```
char name[7] = {'N','I','T','H','I','N','\0'};
```

Nithin VCE

Scanned by CamScanner

→ The Memory Heprin of above defined String in C



Actually, we do not place the null character at the End of a String Contant. The C Compiler Automatically places the '\0' at the End.

→ The C Supports a wide Hange of functions that Manipulate Null terminated Strings

* Strcpy (S1, S2) → Copies String S2 into String S1
* Strcat (S1, S2) → Concatenates String S2 onto the End of String S1.
* Strlen (S1) → Returns the length of String S1.
* StrCmp (S1, S2) → Returns 0 if S1 & S2 are the Same or less than 0 if S1 < S2 or S1 > S2
* Strchr (S1, ch) → Returns a pointer to the first occurance of character ch in String S1.
* StrStr (S1, S2) → Returns a pointer to the first occurance of String S2 in S1.

* Simple programs on Strings (without Functions)

* Concatenation :-

```
#include <stdio.h>
#include <string.h>
int main()
{ char str1[100], str2[100];
  int i, j;
  printf ("Enter the first string");
  gets (str1);
  printf ("Enter the second string");
```

* Reverse :-

```
#include <stdio.h>
#include <string.h>
void reverse (char[], int);
int main()
{ char str1[20];
  int size;
printf ("Enter the String");
scanf ("%s", str1);
size = strlen (str1);
```

Nithin, VCE

Scanned by CamScanner

```
gets (Str2);
for (i=0; Str1[i]!='\0'; i++)
for (j=0; Str2[j]!='\0'; j++, i++)
 {
    Str1[i] = Str2[j];
 }
 Str1[i] = '\0';
printf ("concatenated String = %s", Str1);
Return 0;
}
```

```
Reverse (Str1, 0, size-1);
printf ("String after reversing = %s", Str1);
 Return 0;
}
void Reverse (char Str1[], int index, int size)
{ char temp;
  temp = Str1[index];
Str1[index] = Str1[size-index];
Str1[size-index] = temp;
  if (index == size/2)
    { Return;
    }
 Reverse (Str1, index+1, size);
}
```

* Replace & (Search)

```
#include <stdio.h>
# Include <String.h>
int main()
{
 char Str[100], ch, Newch;
  int i;
 printf ("Enter the String");
 gets (Str);
printf ("Enter the character to Search");
  Scanf ("%c", &ch);
  get char();
  printf ("Enter the new character: ");
  Scanf ("%c", & Newch);
  for (i=0; i<= strlen (Str); i++)
        { if (Str[i] == ch)
            {
              Str[i] = Newch;
            }
      }
    printf ("the final String after replacing = %s", ch, Newch, Str);
    Return 0;
  }
```

Nithin, VCE

Scanned by CamScanner

* Pattern Matching Algorithm :-
Given a String called "pattern p" with 'm' characters & Another String called text with "n" characters where $m \leq n$.
It is required to Search for the pattern String p in the Given text String t.

→ If Search is Successful Return the position of the first occurance of pattern String (p) in text String (t), Otherwise Return -1.

→ There are two pattern Matching Algorithm
  * N-find Algorithm (End indices Algo)
  * Knuth Morris pratt (KMP) Algorithm.

* N-find Algorithm :- The N-find Algorithm Compares the character from pattern with the characters of text String.

→ The End of the String & pattern Arrays are held by "lasts" & "lastp" Respectively.
  * First N-find Compares "String [End Match]" & "pat [lastp]". if they Match, N-find uses 'i' & 'j' to Move through the two Strings until a Mismatch occurs.
    or
    until all of pattern has been Matched.
  * The Variable "Start" is used to Reset 'i' if a Mismat-ch occurs. & "Start" will be Incremented by 1.

* Apply N-find pattern Matching for the foll

Text = a b a b b a a b a a

pattern = a a b

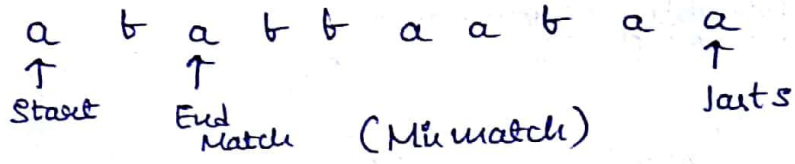
⇒ The pattern will be Represented as

a	a	b
j		lastp

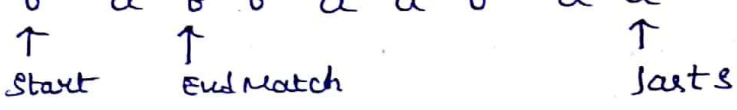
Nithin, VCE

Scanned by CamScanner

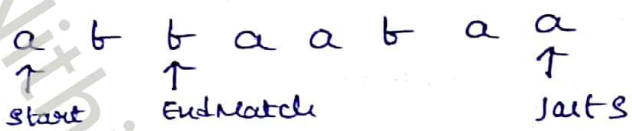
* Iteration 1 :-



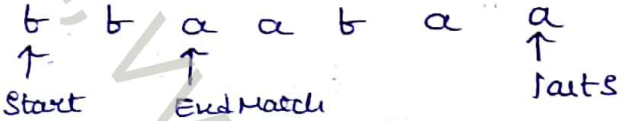
* Iteration 2 :-



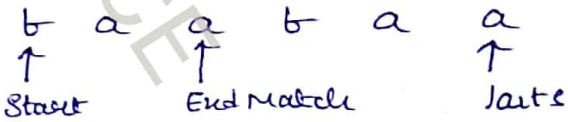
* Iteration 3 :-



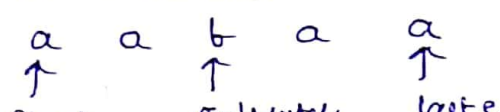
* Iteration 4 :-



* Iteration 5 :-



* Iteration 6 :-



→ The C-function to implement N-find (End-Indices) Algo

```
int nfind (char *String, char *pat)
{ int i, j, start = 0;
  int lasts = Strlen (String) - 1;
  int lastp = Strlen (pat) - 1;
  int Endmatch = lastp;
  for (i=0; Endmatch <= lasts; Endmatch++, start++)
  { if (String [Endmatch] == pat [lastp])
      for (j=0; i=start; j<lastp && &
                String [i] == pat [j]; i++, j++);
    if (j == lastp)
      return start;
  }
  return -1;
}
```

Nithin, VCE

Scanned by CamScanner

* Knuth Morris pratt (KMP) Algorithm :- Uses "prefix" of the pattern & Improves the performance.

→ The Basic Idea behind KMP's Algo is, whenever we detect a Mismatch, we already know some of the characters in the text of Next window.
We take advantage of this information to avoid Matching the characters that we know will anyway Match.

→ prefix code computation for given pattern
  * The prefix code of first character is always '0'.
  * If Index 'i' & 'j' Matches Increment both, otherwise Increment only 'i' & prefix count by "1".

* Apply K.M.P Algo for the given
  * Text :- a b a b c a b c a b a b a b d
  * pattern :- a b a b d

⇒ Firstly, we need Compute prefix-table for given pattern

Index	0	1	2	3	4
pat	a	b	a	b	d
prefix	0	0	1	2	0

→ Next, Apply the prefix code to perform String Matching

Iteration 1 :- a b a b c a b c a b a b a b d

Mismatch occured in place of 'd', we've 'c', upto prefix value = 2 the Match was Successfull so directly Compare the character at Index = 2, If it matches Continue the Matching, otherwise Repeat the step until you reach Index '0'
Once you reach Index j = '0', Increment 'i' by '1'

Nithin, VCE

Scanned by CamScanner

* Iteration 2 :- a b a b c a b c a b a b d
i--i--(2)

Again Make use of prefix-values to move the previously Mat-ched characters, If it not possible, Increment i by 1.

* Iteration 3 :- a b a b c a b c a b a b d
i--i---i-i--(1)
i i i i i

Mismatch occured in place of d, Corresponding prefix-value = 2 so directly start Matching character in index=2 of pattern here it Matching so Continue the procedure

* Apply KMP - String Matching Algorithm for the fol

text: a b c x a b c d a b x a b c d a b c d a b c y

pattern: a b c d a b c y

⇒ Firstly, Generate the prefix-table for given pattern

index	0	1	2	3	4	5	6	7
pat	a	b	c	d	a	b	c	y
prefix	0	0	0	0	1	2	3	0

* Iteration 1 :-

i--i---i-(1)
a b c x a b c d a b x a b c d a b c d a b c y

* Iteration 2 :-

i-i-i-i-i-i-(2)
a b c x a b c d a b x a b c d a b c d a b c y

* Iteration 3 :-

i-i-i-i-i-i-i-(2)
a b c x a b c d a b x a b c d a b c d a b c y
i i i i i i i

Mismatch occured in place of $y \neq d$, the prefix value of previous 'i' =3, so directly start Matching Mismatched character with character at index =3 in pattern.

Scanned by CamScanner

→ The Algorithm of KMP String Matcher

```
* KMP_MATCHER (T, P)
1. n = T.length
2. m = p.length
3. π = Compute_prefix_value (p)
4. q = 0
5. for i = 1 to n
6. while q > 0 & p[q+1] ≠ T[i]        // Mismatch occurred & prefix i.e > 0, start comparison from prefix = index [pattern]
7.          q = π[q]
8. if p[q+1] == T[i]
9.  q = q+1
10. if q == m
11. print "pattern occurs with shift" i-m
```

Nithin, VVCE

Scanned by CamScanner

Module -2

Stack & Queues

There are Certain Situations in Computer Science that one wants to restrict Insertion & deletions so that they can take place only at the beginning or End of list, not in the Middle.

→ Two of such data Structures that are useful
* Stack
* Queue

* **Stack:-** It an Abstract data type (ADT) Commonly used in most programming languages.
A Stack is a list of Elements in which an Element may be Inserted/deleted only at one End called "top" of the Stack.

→ Stacks are known as LIFO (last in first out) lists. As the items can be added or removed only from the top. i.e the last item to be added to a Stack is the first item to be removed.

→ The two basic operations associated with Stacks are
* push
* pop

* **Representation of Stack:** Let us consider a Stack with N Elements capacity, this is called Size of the Stack. The no of Elements to be added should not Exceed Max size > overflow or < underflow

operations on STACK

	Top	STACK contents	output
1) init_stack()	-1	<Empty>	
2) push(a)	0	a	
3) push(b)	1	ab	
4) push(c)	2	abc	
5) pop()	1	ab	c
6) push(d)	2	abd	
7) push(e)	3	abde	
8) pop()	2	abd	e
9) pop()	1	ab	d
10) pop	0	a	b
	-1	<Empty>	a

Scanned by CamScanner

* STACK Implementation :- A Stack can be Implemented using

* Static Array
* dynamic Array
* using linked list

* Stack using Array :- A Stack data Structure can be Implemented using one dimensional Array. But Stack Implemented using Array can Store only fixed no of data values.

→ The Implementation is very Simple, just define a one dimensional Array of Specific Size & insert or delete the values into the Array by using LIFO principle with the help of variable "top" initially top is set to -1.

Ex. #define N 5
int a[N], top = -1;

* push (value)

1. Check whether stack is full (top == Size-1)
2. If it is full then display "Stack full! overflow" & terminate
3. If it is Not Full then Increment top value by one (top++) & set (Stack (top) == value)

* pop ()

1. check whether stack is Empty (top == -1)
2. If it is Empty then display "Stack is Empty! underflow terminate
3. If it is not Empty, then delete Stack(top) & decrement (top)

* display ()

1. check whether Stack is Empty (top == -1)
2. If it Empty display ("Stack is Empty) & terminate

Scanned by CamScanner

3. If it is not Empty, then define a variable x & initialize with top. display stack[i] value & decrement i value by one (i--)

4. Repeat above step until i becomes 0.

* Stack using dynamic Arrays : the issues with Static Stack are

* can run out of space when stack size is too small.
* can waste Memory if stack size set too large.

Use a dynamic Array implementation where Memory for the Stack Array is set dynamically

```
typedef struct {
        int key;
        } element;
element *stack;
MALLOC (stack, sizeof (*stack));
int capacity = 1;
int top = -1;
```

→ Reallocation when stack is full

```
void stackFull()
{
    REALLOC (stack, 2*capacity*sizeof(*stack));
}
```

Scanned by CamScanner

* Application of Stack

* Conversion of Expression → When we write Mathematical Exp in a prgm, we use infix Expression, these Expression will be Converted into Equivalent M/c Instruction by Compiler using Stack.
* Evaluation of Expression → An Arithmetic Expression represented in the form of Either postfix or prefix can be Easily Evaluated.
* Simulation of Recursion → tower of Hanoi problem, problem Involving tree Manipulation.
* parsing → Many Compilers use a Stack for parsing the Syntax of Expression, program blocks Etc. before translating into low level code.
* Conversion of Expression :- An Expression is a legal Combination of operators & operands.

→ operand may be variable like x, y, z or a constant like 5, 4, 6 Etc
operator is a Symbol which Signifies a Mathematical or logical operation bw operands such as +, -, /, * Etc

→ An Expression can be represented using three diff Notations

* Infix
* prefix
* postfix

Scanned by CamScanner

* Infix: It is the form of an Arithmetic Expression in which, we fix (place) the Arithmetic operators in b/w two operands

Ex (A+B) * (C+D)

* prefix: It is the form of an Arithmetic Expression in which, we place the operator before its two operands. The prefix Notation is called as "polish" Notation.

Ex: *+AB+CD

* postfix: It is the form of an Arithmetic Expression in which, we place the operator after its two operands, the postfix Notation is called as "Suffix" notation.

Ex: AB+CD+*

* operator precedence

operator	precedence	Value	Associativity
$, ↑, ^	Highest	4	Right-left
*, /	Next Highest	3	left-Right
+, -	lowest	2	left-Right
#	EOS	1	

* infix-to-postfix: the process of conversion of infix to postfix Expression.

Algorithm

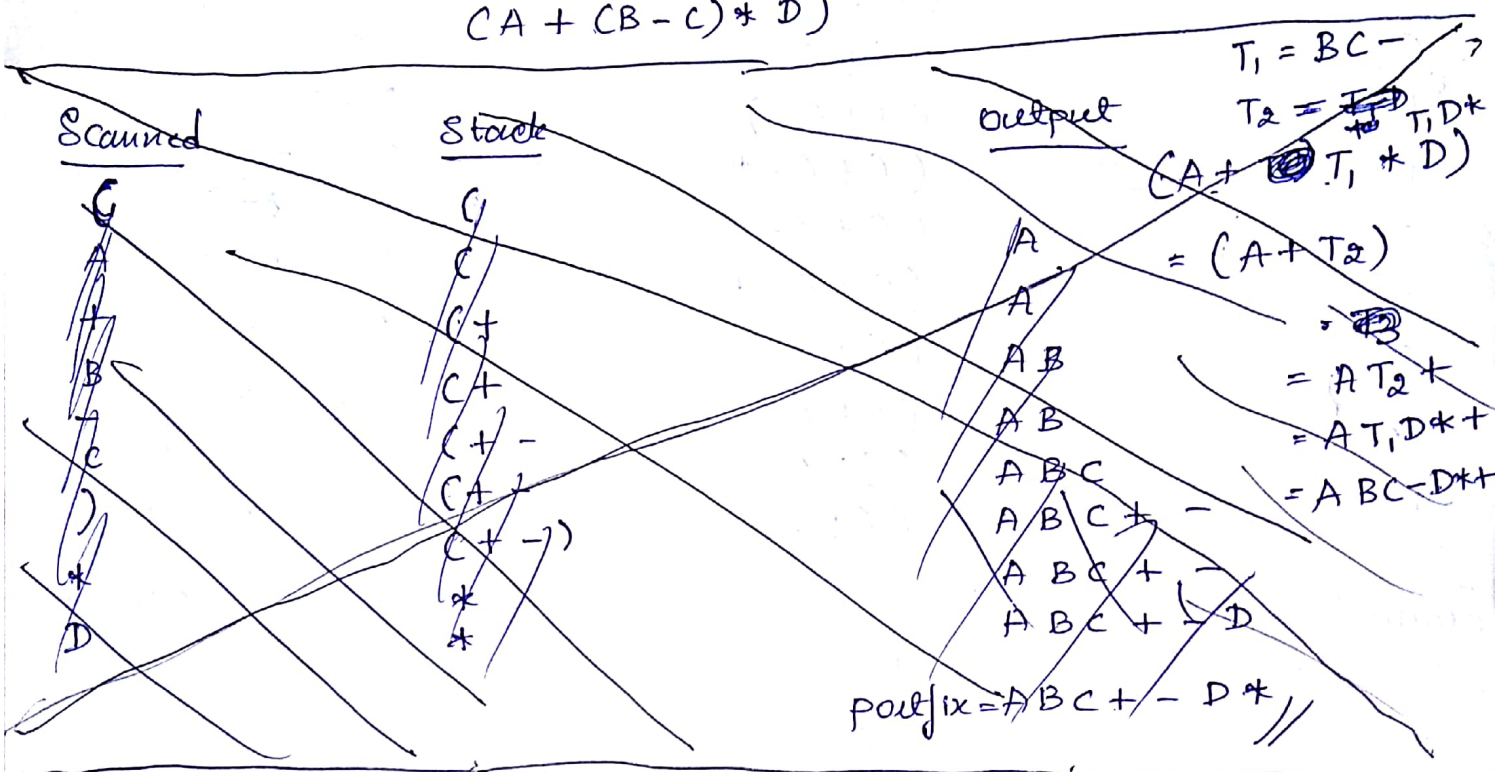
1. Scan the infix Expression from left to right
2. If the Scanned character is operand, output it.
3. Else
   a) If the precedence of Scanned operator is greater than precedence of operator in Stack, push it
   b) Else, pop the operator from Stack until the precedence of Scanned operator is less or Equal to precedence of operator residing on top of the Stack

Scanned by CamScanner

4) if the Scanned character is '(' push it to Stack
5) if the Scanned character is ')' pop & output the Stack until an '(' is Encountered
6) Repeat Step 2-6 until Infix Exp is Scanned.
7) pop & output from stack until it is not Empty.

1) consider the following Exp

(A + (B - C) * D)



Scanned	STACK	output
(	(	
A	(	A
+	(+	A
(	(+(	A
B	(+(	A B
-	(+(-	A B
C	(+(-	A BC
)	(+(-)	A BC -
*	(+*	
D	(+*	A BC - D
)	(+*)	A BC - D*+

Scanned by CamScanner

*((A+(B-C)*D)^E+F)

Scanned	Stack	output
(	(	
(	((	
A	((	A
+	((+	A
(	((+(	A
B	((+(	AB
-	((+(-	AB
C	((+(-	ABC
)	((+)	ABC-
*	((+*)	ABC-D
D	((+*)	ABC-D*
)	(+	ABC-D*+
^	(+^	ABC-D*+
E	(+^	ABC-D*+E
+	(+	ABC-D*+E^
F	(+	ABC-D*+E^F
)		ABC-D*+E^F+

Scanned	STACK	output
[	[	
[	[[	
A	[[	A
+	[[+	
[	[[+[	
B	[[+[	AB
-	[[+[-	
C	[[+[-	ABC
]	[[+[-]	ABC-
*	[[+*	
D	[[+*	ABC-D
]	[[+*]	ABC-D*+
^	[^	ABC-D*+E
E	[^	ABC-D*+E
+	[+	ABC-D*+E^
F	[+	ABC-D*+E^F
]	[+]	ABC-D*+E^F+

Scanned by CamScanner

* $x = a/b - c + d * e - a * c$ $\quad a=4 \quad b=c=2 \quad d=e=3$

Scanned	Stack	output
a		a
/	/	a
b	/	a b
-	/-	a b /
c	-	a b / c
+	-+	a b / c
d	-+	a b / c d
*	-+*	a b / c d
e	-+*	a b / c d e
-	-+*-	a b / c d e * a
a	-+ -	
*	-+-*	
c	-+-*	

a b / c d e * a c * - + -

a b / c d e * a c * - + -

postfix exp	Scanned	op1	op2	result	STACK
42/233*42*-+-	4				4
2/233*42*-+-	2				4 2
/233*42*-+-	/	4	2	2	2
233*42*-+-	2				2 2
33*42*-+-	3				2 2 3
3*42*-+-	3				2 2 3 3
42-+-	*	3	3	9	2 2 9
42*-+-	4				2 2 9 4
2*-+-	2				2 2 9 4 2
*-+-	*	4	2	8	2 2 9 8
-+-	-	9	8	1	2 2 1
+-	+	2	1	3	2 3
-	-	2	3	-1	

-1

Scanned by CamScanner

→ Function to Evaluate postfix exp

```
int Eval()
{
    char Symbol;
    int op1, op2;
    int n=0;          //counter of exp string
    int top=-1;
    token = getToken(&Symbol, &n);
while (token != eos)
    {
      if (token == operand)
            push();
      else {
            op2 = pop();
            op1 = pop();
      switch (token) {
        case plus: push(op1+op2);
                   break;
        case Minus: push(op1-op2);
                   break;
        case divide: push(op1/op2);
                   break;
        case Mul: push(op1*op2);
                   break
      }
    }
    return result;
}
```

Scanned by CamScanner

* A B C − D * + E ^ F + (C A + (B − C) * D) ^ E + F) (4)

A = 6 B = 3 C = 2 D = 5 E = 1 F = 7

Postfix Exp	Scanned	op1	op2	Result	STACK
6 3 2 − 5 * + 1 ^ 7 +	6				6
3 2 − 5 * + 1 ^ 7 +	3				6 3
2 − 5 * + 1 ^ 7 +	2				6 3 2
− 5 * + 1 ^ 7 +	−	3	2	1	6 1
5 * + 1 ^ 7 +	5				6 1 5
* + 1 ^ 7 +	*	1	5	5	6 5
+ 1 ^ 7 +	+	6	5	11	11
1 ^ 7 +	1				11 1
^ 7 +	^	11	1	11	11
7 +	7				11 7
+	+	11	7	18	18

Result = 18

Scanned by CamScanner

* Recursion :-

It a Method of Solving a problem where the Soln depends on Solutions to Smaller Instance of the Same problem. Thus, a recursive function is a function that calls itself during Execution.
This Enable the function to repeat itself Several time to Solve a given problem.

→ The Various type of recursion are
* direct
* Indirect

* direct recursion :- A recursive function that invoke itself is said to have direct recursion.

Ex:
```
int gcd (int a, int b)
{
    if (b == 0)
        return a;
    Else{
        return gcd (b, a % b);
}
```

* Indirect recursion :- A function which contains a call to another function which in turn calls another function & so on & Eventually calls the first function is called as Indirect recursion.

Ex:
```
Void first()
{
  --
  second()
  --
}
Void second()
{
  third();
}
Void third()
{
  first();
}
```

Scanned by CamScanner

* Compute factorial of n :- The operation of recursive factorial function is as follows

→ Start out with some Natural Number N, consider the value of N = 5.

$5! = 5 * 4!$

$4! = 4 * 3!$

$3! = 3 * 2!$

$2! = 2 * 1!$

$1! = 1 * 0!$

$n! = n * (n-1)!$

$0! = 1$ if $n = 0$ // Base case

$$F(n) = \begin{cases} 1 & \text{if } n == 0 \\ n * F(n-1) & \text{if } n > 0 \end{cases}$$

→ Recursive C function to find the factorial of N

```
int fact (int n)
{
    if (n == 0) return 1;
    return n * fact (n-1);
}
```

* GCD - Greatest common divisor (Euclid's Algorithm)

The GCD of two given no is the largest integer that divides both of them. GCD of two no is defined only for +ve no's.

→ The procedure to obtain gcd (30, 24) using Euclid's Alg.

a	b	a % d
30	24	6
24	6	0
(6)	0	

gcd

a	b	a % d
10	6	4
6	4	2
4	2	0
(2)	0	

gcd

Scanned by CamScanner

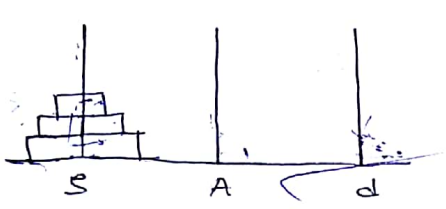
* Tower of Hanoi :-

It a Mathematical puzzle where we have three rods & n disks.

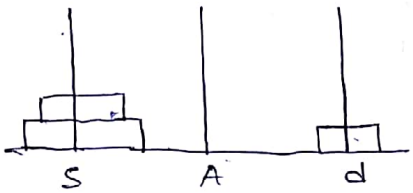
→ The objective of the puzzle is to move the entire disks to another rod, obeying the following simple rules.

* Only one disk can be moved at a time
* A disk can only be moved if it is the upper most disk
* No disk may be placed on top of a smaller disk.

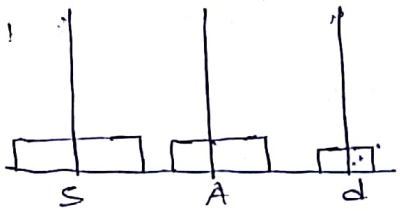
→ Consider the following illustration for 3 disks.



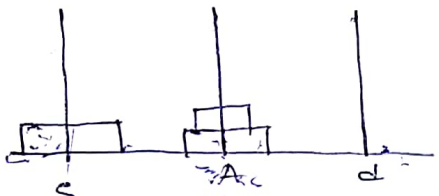
(i)



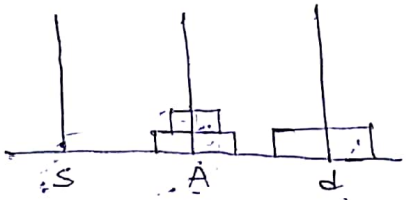
(ii)



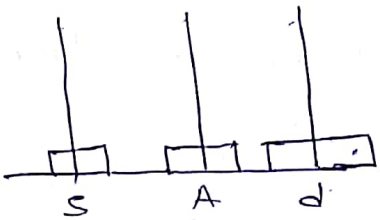
(iii)



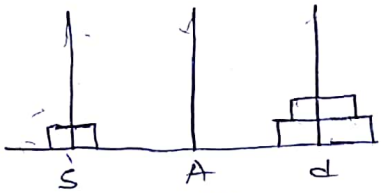
(iv)



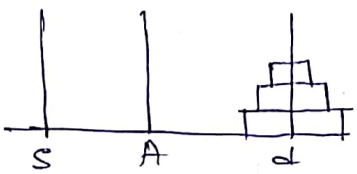
(v)



(vi)



(vii)



(viii)

∴ total no of Moves = $2^n - 1$

$= 2^3 - 1 = 7$

∴ Tower of Hanoi(n) = { S → d if n = 1
Tower (n-1, s, d, a);
Tower (n-1, a, s, d); }

→ Recursive Algo to Solve tower of Hanoi problem.

Scanned by CamScanner

```
Tower_of_Hanoi (n, S, a, d)
{
    if (n == 1)
    {
        Transfer disc from S → d
    }
    Else
    {
        Tower_of_Hanoi (n-1, S, d, a);

        Tower_of_Hanoi (n-1, a, S, d);
    }
```

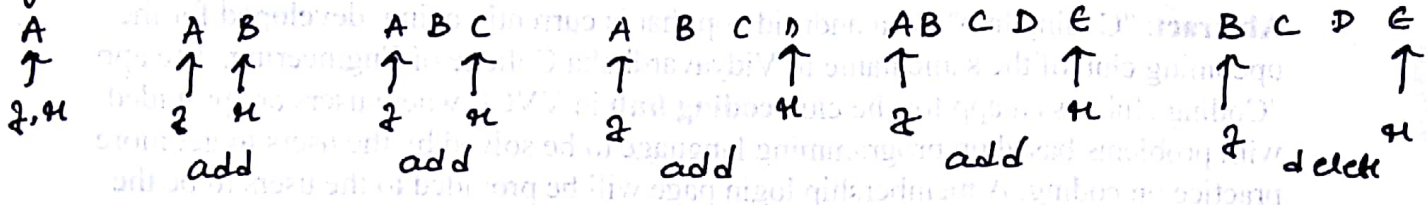
Scanned by CamScanner

* Queue : (FIFO)

A queue is a Linear list of Elements in which deletion can take place only at one End called "front" & Insertion take place only at the other End called "rear".

→ If we insert A, B, C, D & E in order, then A is the first Element deleted from the queue.

→ Figure Illustrates the Sequence of Event



* Representation of Queues & Operations: Queues May be represented by one-way List or Linear array.

→ Two pointer variables "front" contains the location of front Element & REAR containing location of rear Element

→ Front = Null Indicate queue is Empty

→ When an Element is deleted from the queue, the value of front Increased by 1. Front = Front + 1

→ When an Element is added the value of Rear is Increased by 1. REAR = REAR + 1

```
void addq (Element item)
{
    if (rear == MAX-1)
        queue full();
    queue[++rear] = item;
}
```

→ Add an Element to queue

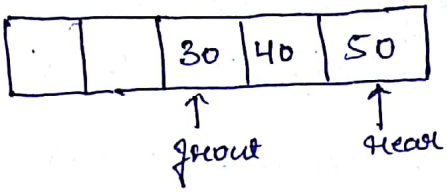
```
Element deleteq()
{
    if (front == rear)
        return Empty();
    return queue[++front];
}
```

→ Delete an Element from queue

Scanned by CamScanner

Circular Queue:-

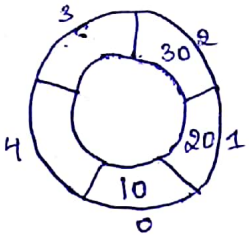
In a Normal queue, we can Insert Elements until queue becomes full, But once queue becomes full, we cannot Insert the next Element Even if there is a Space in front of queue as shown



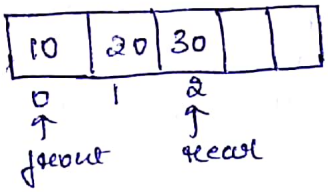
In the above figure; Linear queue, there is a Space in front of queue but we cannot Insert next Element.

→ To overcome this drawback, we Make use of "Circular queue".

→ "Circular queue" is a linear data Structure in which the operation are performed based on FIFO & the last position is Connected back to the first position to Make a Circle.
It is also called "Ring Buffer"



eg:



→ Along with Normal parameters Such as "front", "rear", "MAX" in Linear queue.
In Circular queue, we'll make use of "count" to get the count of Element in queue & "%" operator to calculate the position of Insertion & deletion.

* Insertion:- Firstly, we check whether Sufficient Space is Available in the queue

If (count == MAX)

If queue is full, we'll print "queue overflow" Message.

Scanned by CamScanner

→ Else, we'll calculate, current position of Insertion. & Insert it.

```
rear = (rear+1) % MAX
q[rear] = item;
```

→ Finally, Increment the count by 1

```
count = count + 1;
```

→ The C function to perform Insertion in Circular Queue

```
void Insert (int item)
{ if (count == MAX)
        print ("Queue overflow\n");
  Else
    { rear = (rear+1) % MAX
      q[rear] = item;
      count = count+1;
    }
}
```

*Deletion: Firstly we'll check whether queue is Empty or not by using count

```
if (count == 0)
```

if queue is Empty, we'll print "Queue underflow" Message.

→ Else, we delete the item in "front" & Increment the front by one

```
item = q[front];
front = (front + 1) % MAX
```

→ Finally, decrement the count by 1 & return deleted item.

```
count = count - 1;
```

→ The C function to perform deletion in Circular Queue

```
int delete ( int itemdel)
{
  if (count == 0)
        print ("Queue underflow\n");
  Else
    { itemdel = q[front];
      front = (front+1) % MAX
      count = count-1;
      return (itemdel);
    }
}
```

Scanned by CamScanner

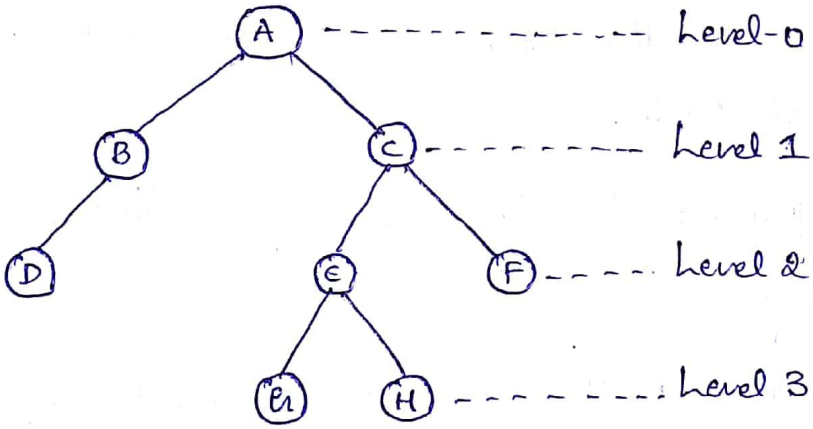
Module-4
Trees

Nithin Kumar
Asst professor
Dept of CS&E
VVCE, Mysuru

Tree is a data structure used to represent hierarchical relat-ionship existing among several data items. Here, each data item is refered as node. Each node may be empty or may be connect-ed to some other nodes.

→ STACK, QUEUE, Linked list etc were linear in nature. In field of Computer Science, we come across many situations where the data are interrelated in hierarchical structures. In this case, a linear repren of data is not possible. The solution for this problem is a data structure called "Tree" which is non-primitive & non-linear data structure.

→ Consider the foll Example of tree



The Tree consists of 'n' nodes & 'e' edges in any tree with 'N' nodes there will be Maximum "N-1" Edges & Every individ-ual Element is called as "Node".

* Basic Terminologies of Tree

* root → is the origin of tree data structure & Every tree must have a root node & there must be only one root node. Ex:- In the above ex tree "A" is root node.

Nithin, VVCE

Scanned by CamScanner

* Edge → Connecting link between any two nodes is called as "Edge". A tree with 'N' nodes consists of "N-1" Edges.

* parent → Node which is predecessor (previous) of any node is called "parent Node"
In General. A node which has branch from it to any other Node is called "parent Node".

* child → Node which is Descendant (Next) of any node is called "child Node"
In General, A node which has a link from its parent is called child nodes.

* Siblings → Nodes which belongs to Same parent are called "Siblings".

* Leaf → the Node which does not have any child is called as "Leaf" Node.

* Internal Node → the Node which has atleast one child is called Internal node.

* Degree of Node → the total no of childrens of a Node is called as "Degree".

* Level of a Node → The root node is said to be at level 0 & the children of root node are at level 1 & the children of nodes which are at level 1 will be level 2 & so on.

* Height of Node → Total no of Edges from leaf node to a particular node in the longest path is called as "Height" of that node.

* Depth of Node → The total no of Edges from root node to a particular node is called "Depth" of that node.

Nithin VGE

Scanned by CamScanner

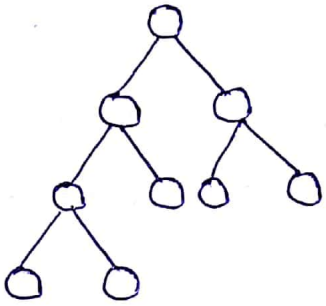
* path → The Sequence of Nodes & Edges from one node to Another node is called as "path".

* Subtree → In a Tree data Structure, Each child from a node forms a Subtree recursively
Every child node will form a Subtree on its parents node.

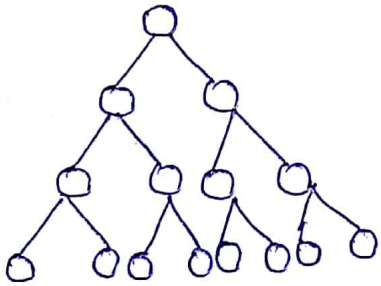
* Binary Tree :-
Binary Tree is a Special type of tree data Stru-cture in which Every node Can have a Maximum of 2 childs. One is known as "Left child" & other is known as "Right child".

Different types of Binary Tree are

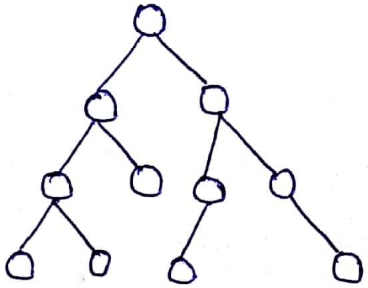
* Strict Binary Tree → Should have Exactly two Childrens or None.



* Complete Binary Tree → All nodes Must have Exactly two childrens at Every level.



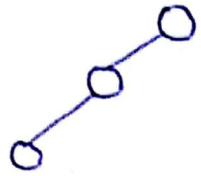
* Almost Complete Binary Tree → Complete binary tree but Completeness property is not followed in last level.



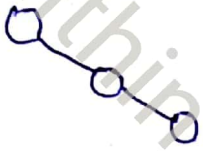
Nitin VCE

Scanned by CamScanner

* Left Skewed binary Tree → A tree with only left Subtree is called left Skewed binary Tree.



* Right Skewed binary Tree → A Tree with only right Subtree is called Right Skewed binary Tree.

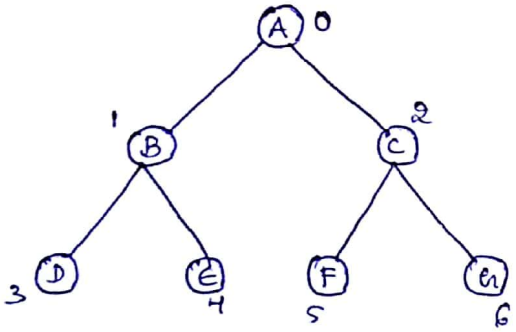


* Binary - Tree Representation :- There are two ways to represent a binary Tree

  * Array representation
  * Linked List representation.

* Array representation → A tree can be represented using an array, which is called "Sequential representation".

→ Consider the following Example



Tree

0	1	2	3	4	5	6
A	B	C	D	E	F	G

Array repn

The nodes are numbered Sequentially from 0. The node with position '0' is considered as the root node.

→ If an index 'i' is 0, it gives position of the root node, Given the position of any other node 'i', "2i+1" gives the position of left child & "2i+2" gives the position of right child.

Nithin, VCE

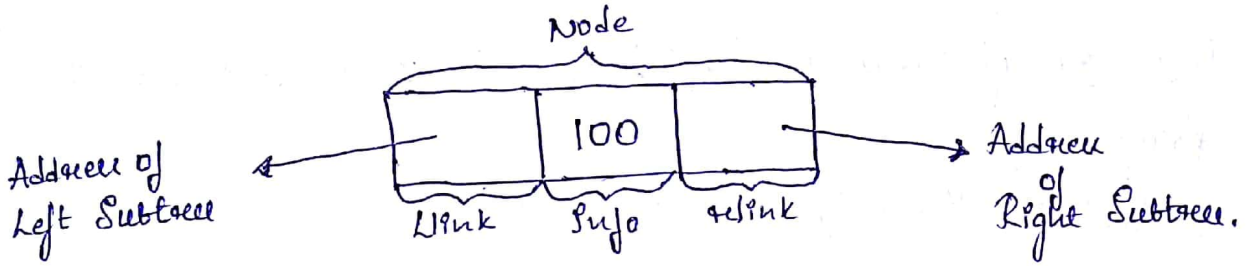
Scanned by CamScanner

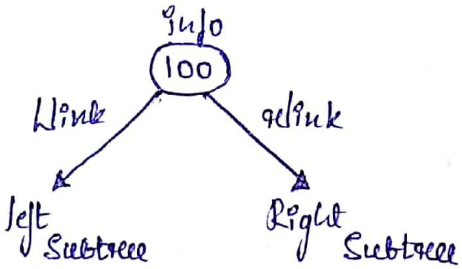
* Linked List representation → In a linked list representation, a Node in a tree has three fields
  * Info → Contains the Actual Information
  * Llink → Contains the Address of Left Subtree
  * Rlink → Contains the Address of Right Subtree

→ A Node can be represented using "Self-referential" Structure

```
Struct Node
{ Int Info;
  Struct Node *llink;
  Struct Node *rlink;
};
Typedef Struct node * NODE;
```

→ The pictorial representation of a typical node in a binary tree





→ A pointer Variable "root" can be used to point to the root node always. If the Tree is Empty, the pointer Variable "root" points to NULL. Indicating the tree is Empty.

* Advantages of Array repsn
  → Fast Accen
  → Easy for Implementation
  → Good for Complete binary Trees

Nithin, VCE

Scanned by CamScanner

* Disadvantage of Array repn
→ Waste Memory for Skewed trees
→ Implementation of operation requires rearranging of Elements.

* properties of Binary Trees-

* prove that max no of nodes in a BT of depth k is $2^k - 1$

⇒ Total no of nodes = $2^0 + 2^1 + 2^2 \ldots + 2^i$

The above sequence is in geometric progression

$= (2^{i+1} - 1)/(2-1)$

$= 2^{i+1} - 1$

$= 2^n - 1$ (where $i+1 = n$ = height of tree)

∴ Max no of nodes in a BT of depth $k = 2^k - 1$ //

* Max no of nodes on level i of a BT is 2^{i-1}, $i >= 1$ or 2^i, $i >= 0$

⇒ proof :-

Root is the only node on level $i = 1$

Hence Max no of nodes on level $i = 1$

$\Rightarrow 2^{i-1} = 2^0 = 1$

→ let I be an arbitary positive Integer > 1

Assume that max no of nodes on level $i-1$ is 2

→ we know that max no of nodes on level $i-1 - 2^{i-2}$ by Induct -ion hypothesis

we know that Each node in a BT max degree is 2

∴ Max no of nodes on level i = twice the max no of nodes on level $i-2$

i.e 2^{i-1} //

Nithin VCE

Scanned by CamScanner

* prove that no of leaf nodes = no of nodes of degree-2 nodes for any non-Empty Binary Tree T, If N_0 is the no of leaf nodes & N_2 no of nodes of degree 2 then $N_0 = N_2 + 1$

⇒ proof :- let N_1 be the no of nodes of degree 1, let 'N' be the total no of nodes

Since all nodes in T are atmost of degree 2 we have

$$N = N_0 + N_1 + N_2 \text{ ---------- } (1)$$

If we count no of branches in a BT we see that Every node Except the root has a branch leading into it.

→ If B → No of branches then, $N = B + 1$ (because all branches setup from a Node of degree 1 or 2)

$$\therefore B = N_1 + 2N_2$$

$$N = B + 1$$

$$N = N_1 + 2N_2 + 1 \text{ -------- } (2)$$

from (1) & (2)

$$N_0 + N_1 + N_2 = N_1 + 2N_2 + 1$$

$$N_0 = N_2 + 1 \text{ //}$$

* Binary Search Tree (BST)

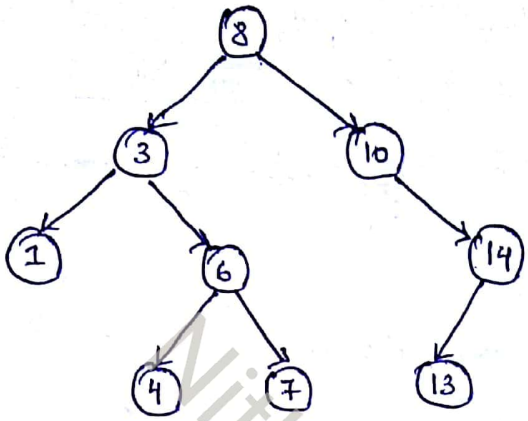
Is a rooted binary tree whose Internal nodes Must follow the properties Mentioned below

* The left Subtree of a node Contains only nodes with key lesser than the node's key
* The Right Subtree of a node Contains only nodes with key Greater than the nodes key
* The left & Right Subtree Each Must also be a BST.

Nithin VCE

Scanned by CamScanner

→ Consider the foll Example of Binary Search tree (BST)



→ BST divides all its Sub-tree into two Segments

* Left Subtree
* Right Subtree

can be defined as

left_Subtree (keys) ≤ Node (key) ≤ Right_Subtree (keys)

* Binary Search Tree Construction :- The construction of BST consists of Three Steps.

  * Step 1 → place the first value in the given list as root node.

  * Step 2 → for next value in the given list of values, compare with root value
    * If greater than root, place it towards "Right"
    * If lesser than root, place it towards "Left"

  * Step 3 → Repeat Step2 for all values apart from first value (root).

* Construct a Binary Search Tree by Inserting the foll Sequence of numbers

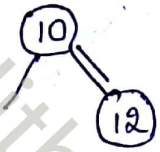
  10, 12, 5, 4, 20, 8, 7, 15 & 13

Nithin VCE

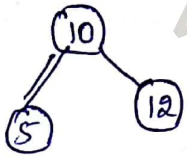
Scanned by CamScanner

⇒ The first value is '10', it becomes the root

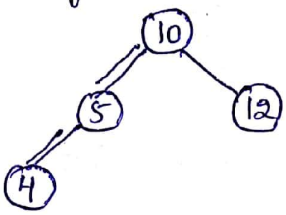
→ The next value is 12, which is greater than 10, so place it in 'right'



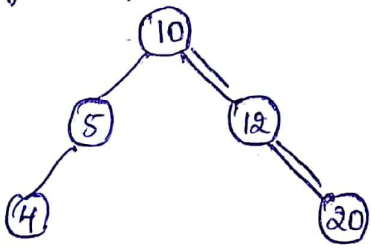
→ The next value is 5, which is lesser than 10, so place it in "Left"



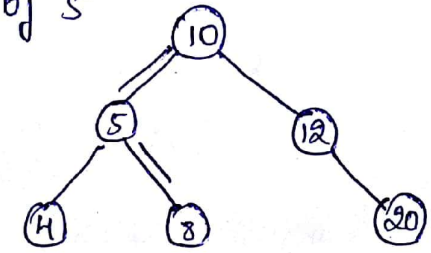
→ The next value is 4, which is lesser than 10, & when compared with 5, Again lesser, so place it in 'Left' of 5



→ The next value is 20, which is greater than 10, so take right Sub-tree, when compared with '12' Again it's greater, so place it in 'right' of 12



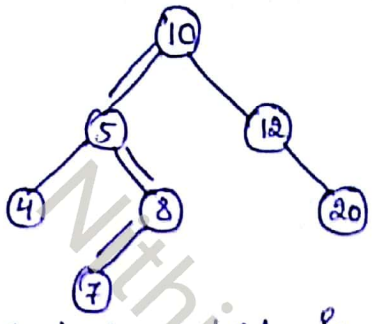
→ The next value is 8, which is lesser than 10, so take Left-Sub-tree, when compared with '5' it's greater than 5 so place it in 'right' of 5



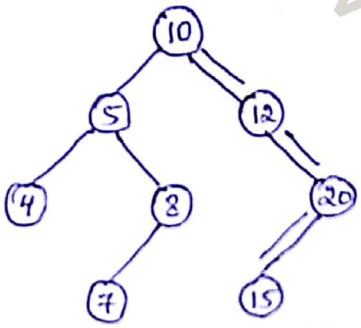
Nithin, VCE

Scanned by CamScanner

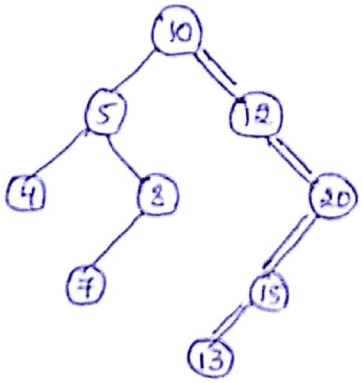
→ The next value is '7', which is lesser than '10', so take left Sub-tree, when compared to 5, it greater than 5 so take right Sub-tree, Again when compared with '8' it lesser, so place it towards 'left'.



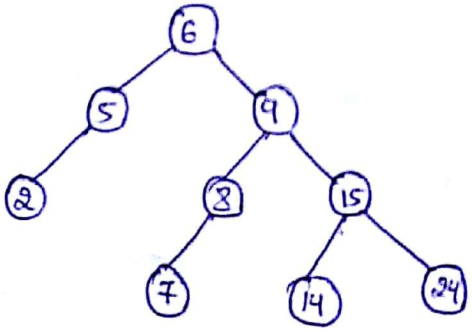
→ The next value is '15', which is greater than 10, Again in right-Sub-tree it greater than 12, but when compared with '20' it lesser, so place it towards 'left' of '20'.



→ The next value is '13', which is greater than 10, Again it greater than 12, but lesser than 20 & 15, so place it towards 'left' of '15'



* Construct the BST for :- 6, 9, 5, 2, 8, 15, 24, 14, 7



Note:- If duplicate values found, ignore the duplicates while Construct

Nithin VCE

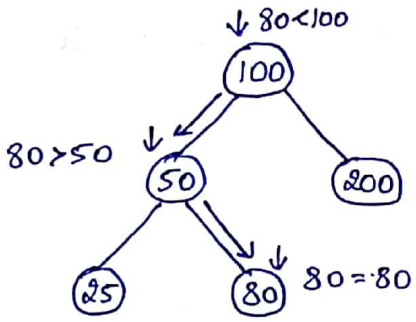
Scanned by CamScanner

* Binary Search Tree operations :- The foll are the basic operations of BST

  * Search
  * Insertion
  * Traversal
    - pre-order
    - post-order
    - In-order

* Search operation :- Whenever an Element is to be Searched in BST, Start Searching from root node. Then
  * If the root = Element, Search is Succesful.
  * Else, If the data is less than the key value, Search in "Left - Subtree"
  * Else, If the data is greater than the key value, Search in "Right - Subtree".

→ Consider the foll Example BST & Item to be Searched is "80"



Firstly, the 'Item = 80' is Compared with "root = 100", Item is lesser than root. So traversed towards Left Subtree, Again Item is compared with '50', Item is greater than root. So traversed towards right Subtree, Again Item is Compared with '80'. Succesful Search the Item found.

→ The function to perform Search operation in BST

```
Node Search (int Item, NODE root)
{
  NODE Cur;
  If (root == Null)
        Return Null;
```

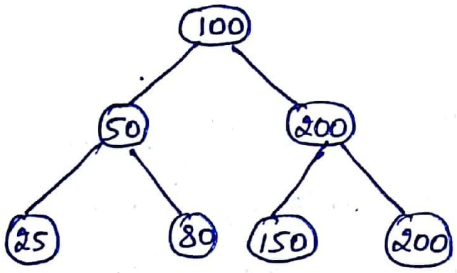
Nithin, VVCE

Scanned by CamScanner

```
Curr = root;
while (Curr != Null)
{   if (item == Curr → info)   // Comparing with root
        return Curr;
    if (item < Curr → info)   // Traverse left Subtree
        Curr = Curr → Llink;
    Else
        Curr = Curr → rlink;  // Traverse right Subtree
}
return Null;
}
```

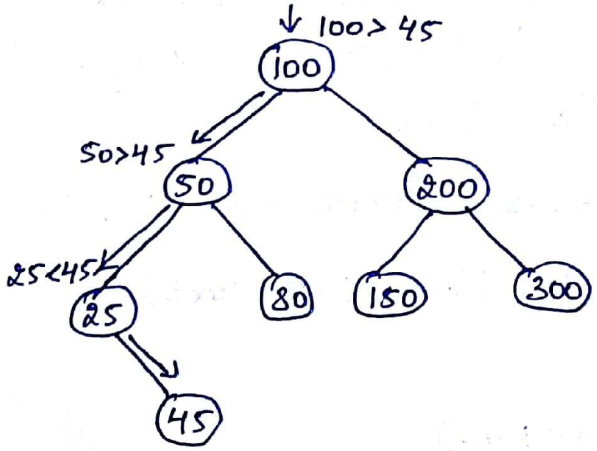
* Insert operation :- Whenever an Element is to be Inserted, first locate its proper location, Start Searching from the root node then
  * If the data is Less than the key value (root). Search for Empty location in left Subtree & Insert the data.
  * Otherwise, Search for the Empty location in right Subtree, & Insert the data.

→ Consider the foll Example



Item to be Inserted = 45

⇒



Nithin, VCE

Scanned by CamScanner

→ The C function to insert the node in BST

```
Node insert (int item, NODE root)
    {
      Node temp, cur, prev;
       temp→info = item;
       temp→llink = Null;
       temp→rlink = Null;
  if (root == Null)
                return temp;
     prev = Null;
     cur = root;
while (cur != Null)
    {   prev = cur;
      if (item < cur→info)
              cur = cur→llink;
        Else  cur = cur→rlink;
    }
 if (item < prev→info)
          prev→llink = temp;
       Else
          prev→rlink = temp;
       return root;
}
```

* Traversal :- Traversal is a process to visit all the nodes of a tree & may print their values too. We cannot randomly access a node in a Tree.

→ There are three ways which we use to traverse a tree

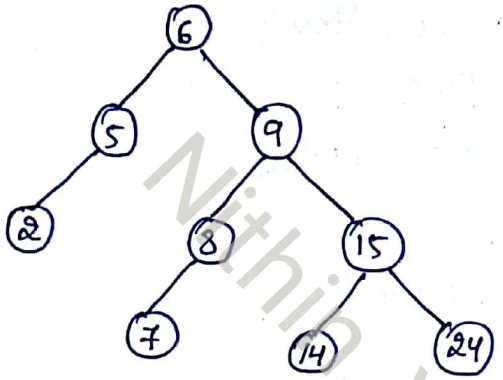
* pre-order → N-L-R - ~~Anti~~-clockwise from "root"
* post-order → L-R-N - Anti-clockwise from "Left"
* In-order → L-N-R - clockwise from "Left"

Nitin, VCE

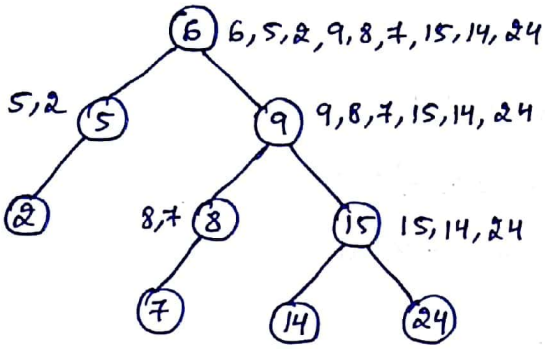
Scanned by CamScanner

* Construct a BST on given values & Show Traversals using preorder, postorder & Inorder Traversals.

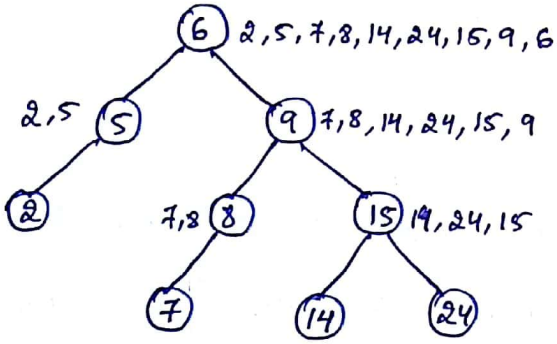
⇒ 6, 9, 5, 2, 8, 15, 24, 14, 7



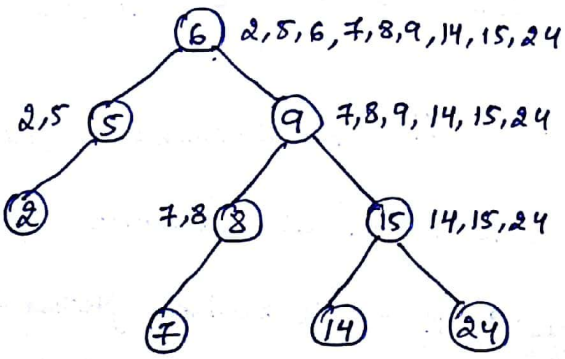
* preorder – N-L-R (Anti-clockwise) = 6, 5, 2, 9, 8, 7, 15, 14, 24 //



* postorder – L-R-N (Anti-clockwise) = 2, 5, 7, 8, 14, 24, 15, 9, 6 //



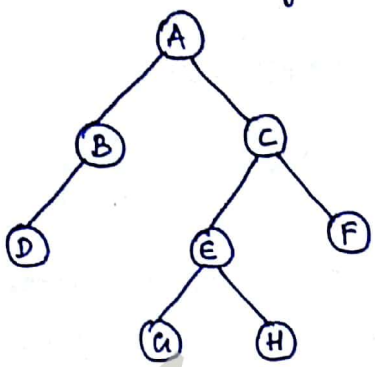
* Inorder = L-N-R (clockwise) = 2, 5, 6, 7, 8, 9, 14, 15, 24 //



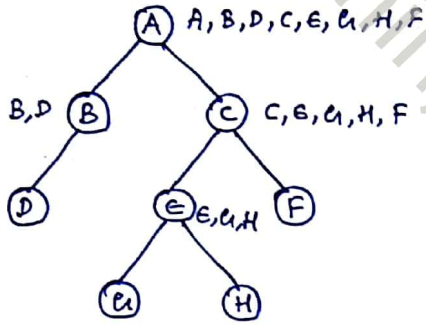
Nithin, VCE

Scanned by CamScanner

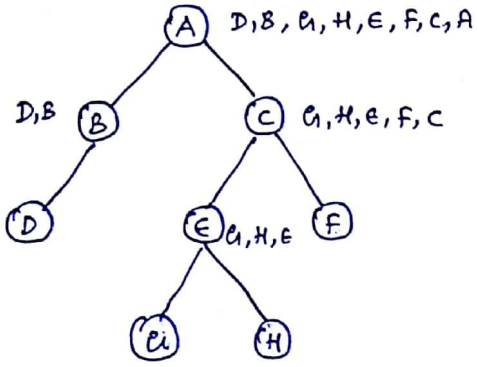
* Traverse the Given tree using Inorder, preorder & postorder Traversal



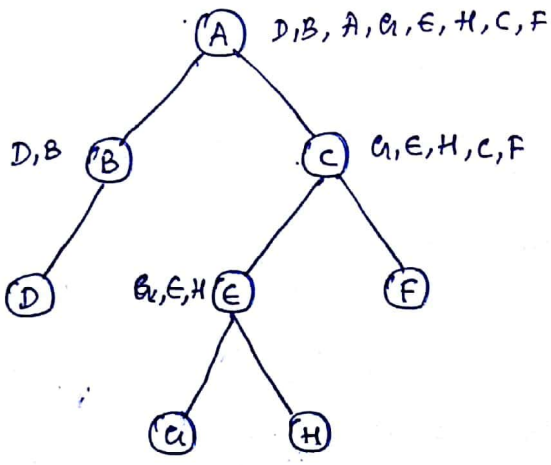
* preorder - N-L-R (Anti-clockwise) = A, B, D, C, E, G, H, F //



* postorder - L-R-N (Anti-clockwise) = D, B, G, H, E, F, C, A //



* Inorder - L-N-R (clockwise) = D, B, A, G, E, H, C, F //



Nithin, VCE

Scanned by CamScanner

→ Recursive functions to perform traversal

* preorder →
```
void preorder (NODE root)
{
    if (root == NULL) return;
    printf ("%d\t", root->info); // print root info
    preorder (root->llink); // Traverse left subtree
    preorder (root->rlink); // Traverse right subtree
}
```

* postorder →
```
void postorder (NODE root)
{
    if (root == Null) return;
    postorder (root->llink); // Traverse left subtree
    postorder (root->rlink); // Traverse right subtree
    printf ("%d\t", root->info); // print root info
}
```

* Inorder →
```
void Inorder (NODE root)
{
    if (root == Null) return;
    Inorder (root->llink); // Traverse left subtree
    printf ("%d\t", root->info); // print root info
    Inorder (root->rlink); // Traverse right subtree
}
```

Note:- prepare for lab program-10, which covers all three Basic operation such as Insert, Search & Traversal.

* Additional operation on BST:- The various operation on BST are shown below

* Finding Max item in tree
* Finding Min item in tree
* Finding Height of tree
* Counting the no of Nodes
* Counting the no of Leaf Nodes

Nithin VCE

Scanned by CamScanner

* Finding Max Item in tree :- A Node with Max value is obtained by traversing & obtaining the rightmost node in the tree.

```
NODE maximum (NODE root)
{
    NODE cur;
    if (root == NULL)
        return root;
    cur = root;
    while (cur -> rlink != Null)
    {
        cur = cur -> rlink;
    }
    return cur;
}
```

* Finding Min Item in tree :- A Node with Least value is obtained by traversing & obtaining the left-most node in the tree. If there is no left Subtree then return root itself as the node Containing an item with least value.

```
NODE Minimum (NODE root)
{
    NODE cur;
    if (root == NULL)
        return root;
    cur = root;
    while (cur -> Llink != Null)
    {
        cur = cur -> Llink;
    }
    return cur;
}
```

* Height of tree :- Height of the tree is Nothing but Max level in a tree.

Recursive function of Height = $\begin{cases} -1 & \text{if root = NULL} \\ 1 + \max(\text{height(root} \to \text{llink}), \text{height(root} \to \text{rlink})) \end{cases}$

```
int height (NODE root)
{
    if (root == Null) return -1;
    return 1 + max (height (root -> llink), height (root -> rlink));
}
```

Nithin VCE

Scanned by CamScanner

* Count Nodes in a tree :- The no of Nodes in a tree is obtained by traversing the tree in any of the traversing technique & Increment the Counter whenever a node is visited.

```
Void count_Nodes (Node root)
{ if (root == NULL) return;
    count_nodes (root → llink);
    count++;
    count_nodes (root → rlink);
}
```

* Count leaves in a tree :- A leaf or a terminal node is one, which has an Empty left & right child. Whenever a leaf is Encountered update Count by one.

```
Void count_leaf (NODE root)
{ if (root == NULL) return;
    count_leaf (root → llink);
if (root → llink == NULL && root → rlink == NULL) count++;
    count_leaf (root → rlink);
}
```

* Iterative Traversal :- Recursive way of tree traversing calls itself Recursively to compute traversal order.
Using "STACK" we can traverse a tree without recursion.

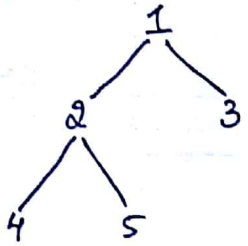
→ The Algorithm for Traversing Binary tree using STACK is

1. Create an Empty Stack S
2. Initialize Current node as root
3. push the Current Node to S & Set Current = Current → left until Current is NULL
    a) pop the top item from STACK
    b) print the popped item, set Current = popped_item → right
    c) Repeat

Nithin, VVCE

Scanned by CamScanner

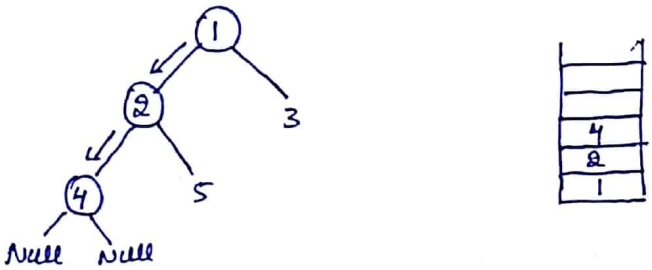
→ Consider the foll Example



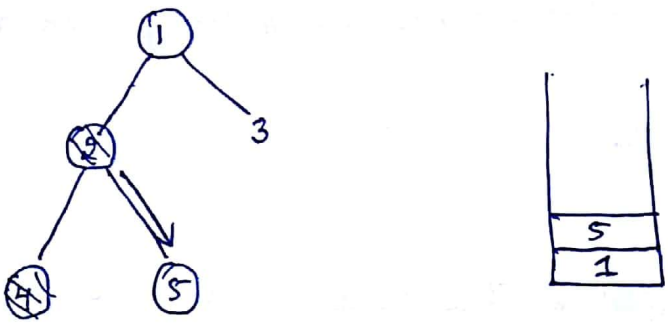
* Step 1 :- Create an Empty Stack S = Null

* Step 2 :- Initialize root = Current & push it to Stack & change Current = Current → llink

* Step 3 :- push the Current Item to Stack till Current = Null



* Step 4 :- Once Current is Null, pop the Item from Stack & display in o/p & change "Current = Current → rlink"

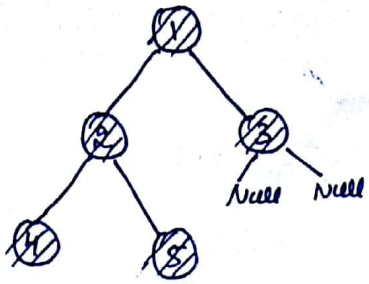


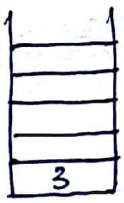
o/p :- 4, 2

* Step 5 :- Check for Subtree in that "rlink" if there is Subtree Again repeat Same procedure, otherwise pop.

Nithin, VCE

Scanned by CamScanner





O/p:- 4, 2, 5, 1

∴ pop = 3 i.e O/p:- 4, 2, 5, 1, 3 //

Once Current = Null & STACK is Empty, you're done with traversal.

→ C function to implement Iterative Inorder Traversal

```
Void Inorder (NODE root)
{
    NODE cur, S[20];
    int top = -1;
    if (root == NULL)
    {
        printf ("Tree is Empty\n");
        return;
    }
    cur = root;
for (;;)
{
    while (cur != Null)
    {
        S[++top] = cur;
        cur = cur->llink; // Traverse towards left
    }
    if (top != -1) //check whether STACK is Empty
    {
        cur = S[top--]; //pop
        printf ("%d", cur->info);
        cur = cur->rlink; //change cur to right link
    }
    Else
        return;
}
}
```

* Level order Traversal :- The node in a tree are numbered starting with the root on level 0, continuing with nodes on level 1, level 2 & so on.

Nithin, VVCE

Scanned by CamScanner

→ Nodes on any level are numbered from "left" to "right". Visiting the nodes using the ordering suggested by node numbering is called "level order Traversing".

→ For level order Traversing, we'll make use of Queue (FIFO)

* Algorithm for level order Traversal

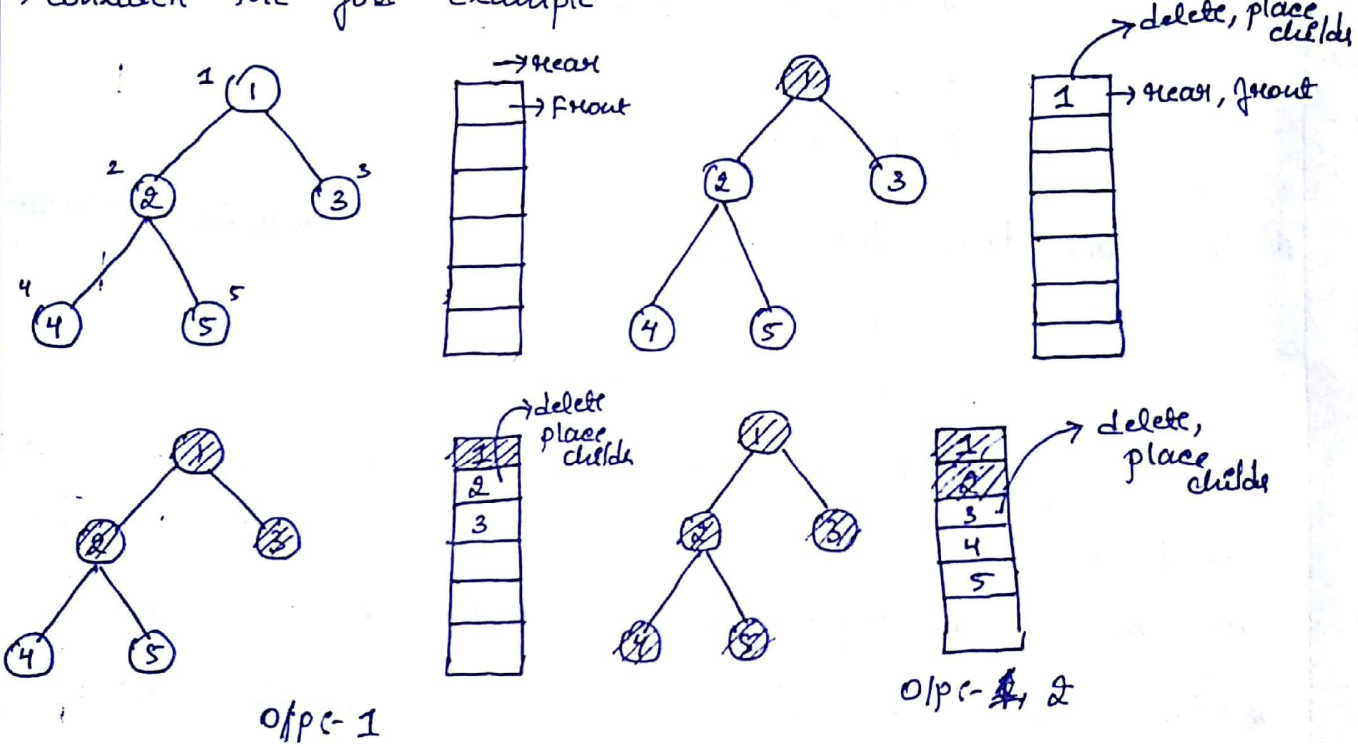
* Step 1 → Insert the root node into Queue

* Step 2 → If node Inserted has a left Subtree & right Subtree Insert that nodes to Queue

Once after Insertion of child nodes, delete the parent node from Queue & display in o/p.

* Step 3 → Repeat the above Steps as long as Queue is not Empty.

→ Consider the foll Example



→ As, there is no child nodes for 3, 4 & 5, pop them one by one display in o/p:- 1, 2, 3, 4, 5.

→ The C function to Implement level order Traversal

```
Void level-order (NODE root)
{
    NODE a[MAX-QUEUE], Cur;
```

Nithin VCE

Scanned by CamScanner

```
int front = 0, rear = -1;
q[++rear] = root;
while (front <= rear)
    {
        cur = q[front++];
        printf ("%d", cur->info);
        if (cur->llink != NULL)
            q[++rear] = cur->llink;
        if (cur->rlink != NULL)
            q[++rear] = cur->rlink;
    }
}
```

* Constructing Binary Tree using Traversal pairs :- The Binary Tree can be Constructed using any two Traversal pairs such as

  * Inorder & preorder
  * Inorder & postorder
  * postorder & preorder

* Construct tree from given Inorder & preorder Traversal

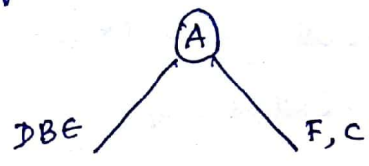
  Inorder - D, B, E, A, F, C

  preorder - A, B, D, E, C, F

⇒ * Firstly, find the first node of pre-order, the first node is "root node", In the above Ex 'A', we know this value is root as root always appear in the first place of preorder.

* Then, we search for 'A' in in-order to find left & Right Subtree of root.

  Everything on left of 'A' in Inorder is left-Subtree & Everything on right of 'A' is right-Subtree.



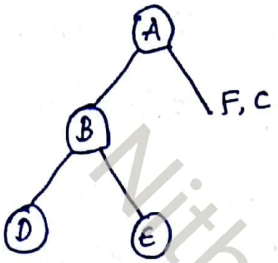
Nithin VCE

Scanned by CamScanner

→ Again, we Repeat the Same Steps, till we End-up with Sing -leton - values.

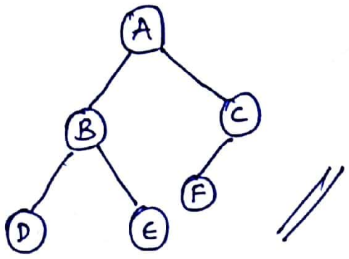
Inorder [D, B, E] = preorder [B, D, E]

∴ Root = B



→ In-order [F, C] = preorder [C, F]

∴ Root = C

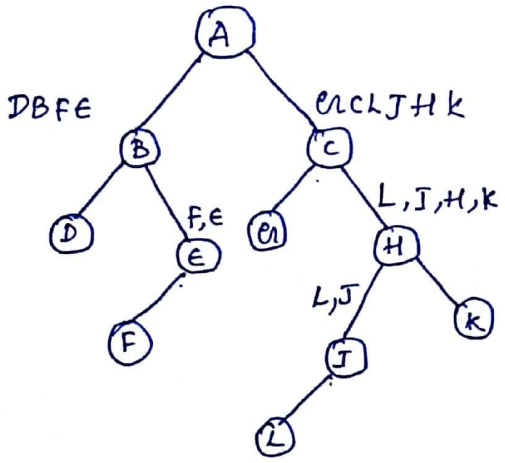


* Construct the Binary Tree using the foll

pre-order = A, B, D, E, F, C, G, H, J, L, K

In-order = D, B, F, E, A, G, C, L, J, H, K

⇒ As, we know the first node in preorder = root = A then Search for 'A' in In-order & perform Split.



In [DBFE] = pre [BDEF]

∴ root = B

In [FE] = pre [EF]

∴ root = E

In [GCLJHK] = pre [CGHJLK]

∴ root = C

In [LJHK] = pre [HJLK]

∴ root = H

In [L, J] = pre [J, L]

Nithin, VVCE

Scanned by CamScanner

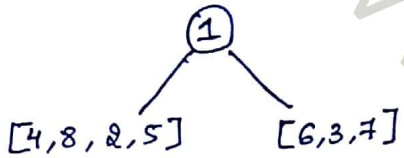
* Construction a Binary Tree from Inorder & postorder

Inorder - 4, 8, 2, 5, 1, 6, 3, 7

postorder - 8, 4, 5, 2, 6, 7, 3, 1

⇒ * First, find the "last node" of post-order. The last node in above Ex is '1', we know this value is root as root always appear in the End of postorder traversal.

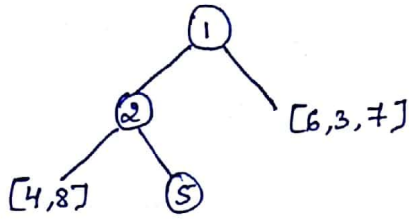
* Then, we Search '1' in In-order to find left & Right Sub tree of root, Everything on left of '1' in In-order is in left Subtree & Everything on right is in right Subtree.



→ Again, Repeat the procedure

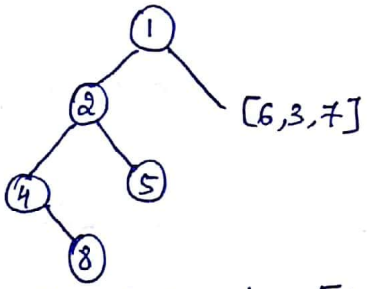
Inorder [4,8,2,5] = postorder [8,4,5,2]

∴ Root = 2

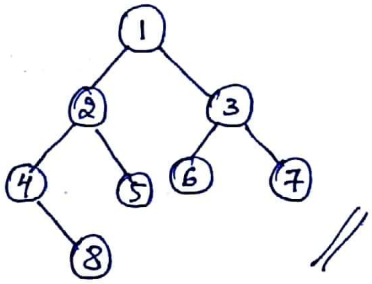


→ In-order [4,8] = postorder [8,4]

∴ Root = 4



→ Inorder [6,3,7] = postorder [6,7,3] ∴ Root = 3



//

Nithin, VCE

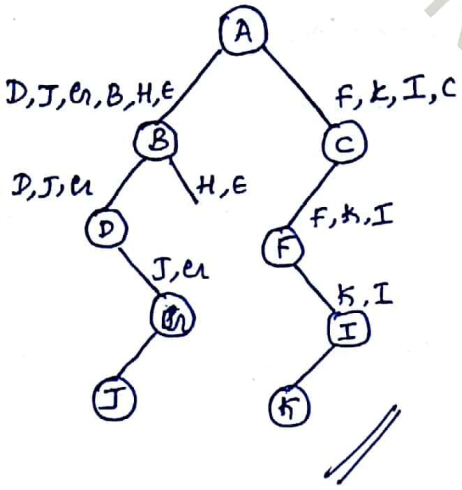
Scanned by CamScanner

* Construct a Binary Tree from Inorder & preorder

Inorder : D, J, er, B, H, E, A, F, K, I, C

postorder : J, er, D, H, E, B, K, I, F, C, A

⇒ As, we know that last node in postorder = root = A, then Search for 'A' in Inorder & perform split.



In [D,J,er,B,H,E] = post[J,er,D,H,E,B]
∴ root = B

In [D,J,er] = post [J,er,D]
∴ root = D

In [J er] = post [J,er]
∴ root = er

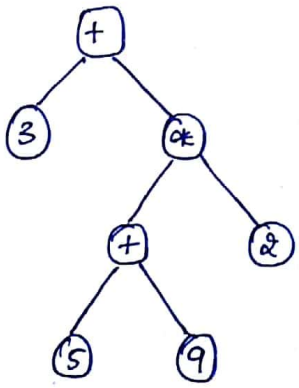
In [F,K,I,C] = post [K,I,F,C]
∴ root = C

In [F,K,I] = post [K,I,F]
∴ root = F

In [K,I] = post [K,I]
∴ root = I

* Expression Tree :- is a binary tree in which Each "Internal node" Corresponds to "operator" & Each "leaf node" Corresponds to "operand".

→ Consider the foll Example Expression 3+((5+9)*2)



Nithin, VCE

Scanned by CamScanner

→ Binary Expression tree can represent two types of Expressions.

* Algebric Expression
* Boolean Expression

→ If you Apply traversal procedure on Binary Expression tree the results are

* Inorder → Infix Expression
* preorder → prefix Expression
* postorder → postfix Expression

→ Using postfix Expression, we can construct Expression tree by the foll steps.

* Step 1 :- Reading postfix Expression. (If the given Expression is infix, convert it to postfix).

* Step 2 :- Read only one Symbol at a time.

* Step 3 :- If Symbol is

  * operand : Create a node & push pointers onto Stack
  * operator : pop 2 pointers & form a new Tree
    → first popped = Right child
    → Second popped = left child

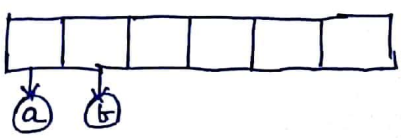
  then push pointer of this new tree onto STACK.

* Construct Expression tree for the foll Expression

ab+cde+**

⇒ Firstly, check whether the given Expression is in postfix form. If it is not in postfix form convert it.

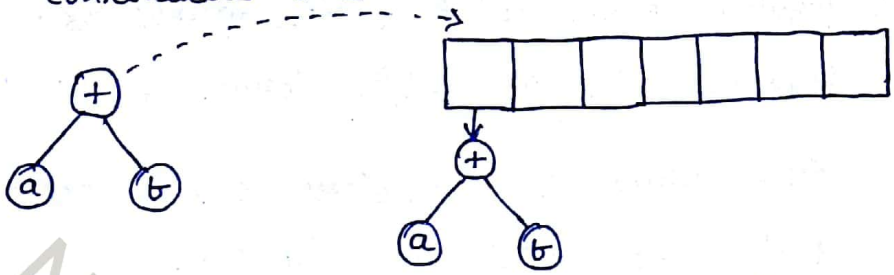
The above Expression is in postfix form, so I read first two operands & place it on STACK.



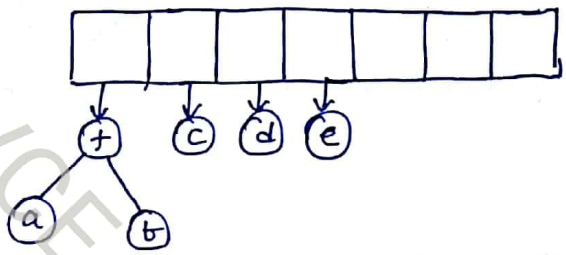
Nithin, VCE

Scanned by CamScanner

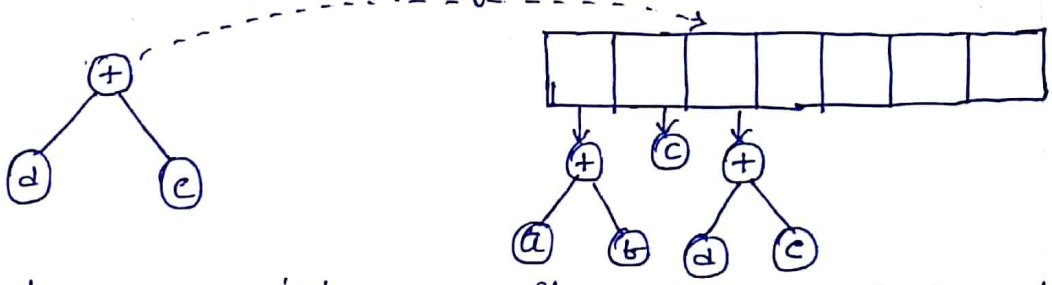
→ The Next Character is '+' Operator), so pop 2 pointers from STACK & place operator in b/w them for tree construction & push the newly constructed tree



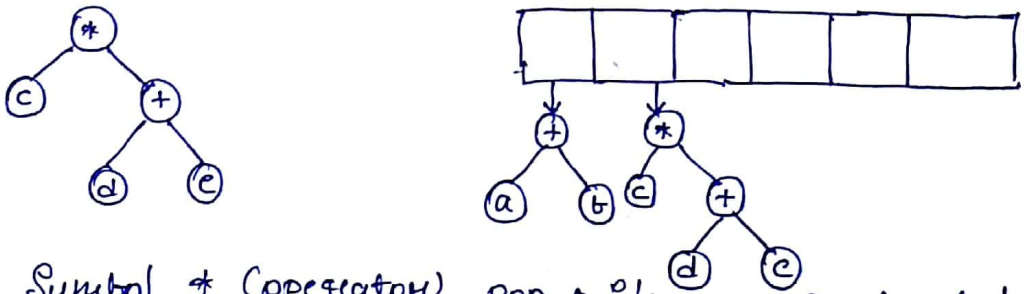
→ The Next characters are 'c', 'd', & 'e' Operands) just push it onto STACK one by one.



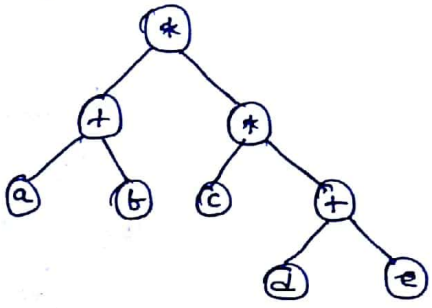
→ Next, we found operator '+', pop 2 items from STACK & place '+' in b/w them, push newly constructed tree



→ Next, we found operator '*', pop 2 items & construct new tree & push.



→ Next, the last Symbol * Operator), pop 2 items & construct tree.



//

Nithin, VCE

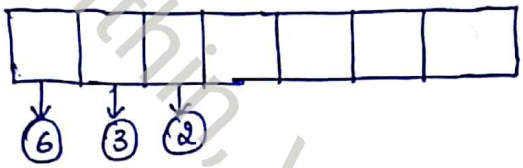
Scanned by CamScanner

* Construct a Binary Tree for following postfix Exp

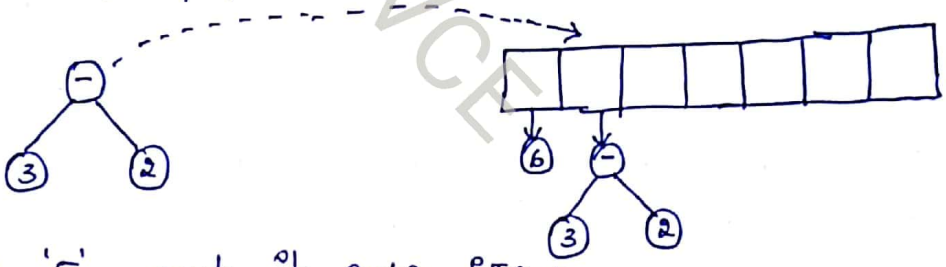
6 3 2 - 5 * + 1 $ 7 +

⇒ Firstly, check whether the given Exp is in postfix, the above Expression is in postfix form. so No problem.

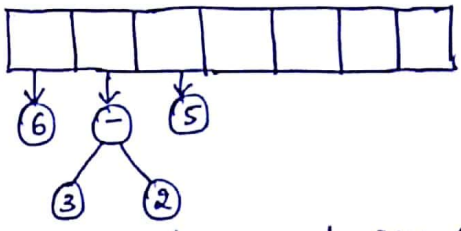
→ The First 3 characters are operand, so directly we can push onto STAck one by one.



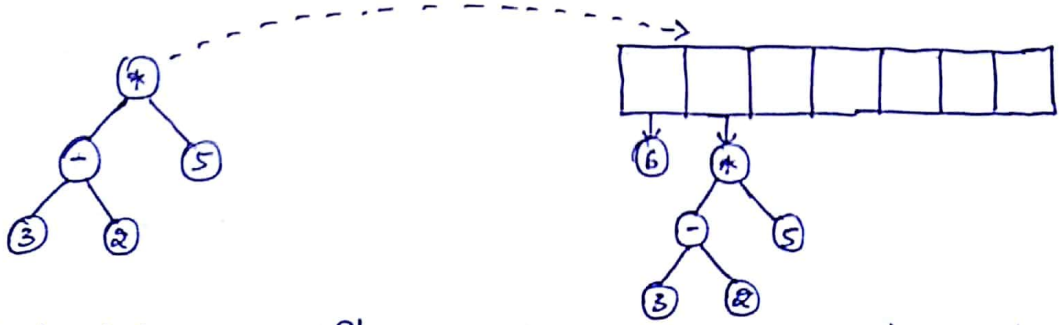
→ Next operator '-', pop 2 Items & Construct then push result.



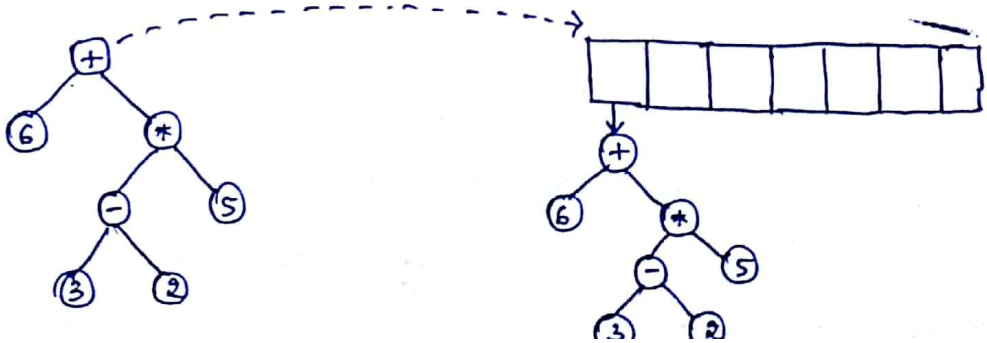
→ Next Symbol is '5', push it onto STAck.



→ Next Symbol is '*', pop 2 Items & Construct then push result.



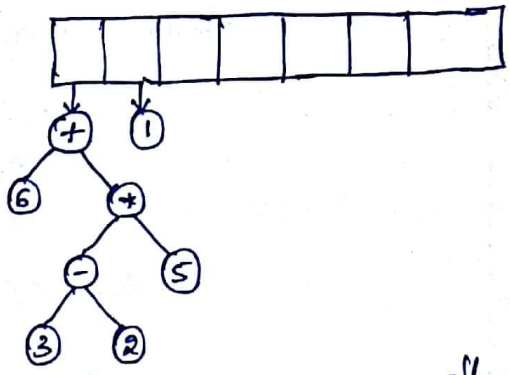
→ Next Symbol is '+', pop 2 Items & Construct then push results.



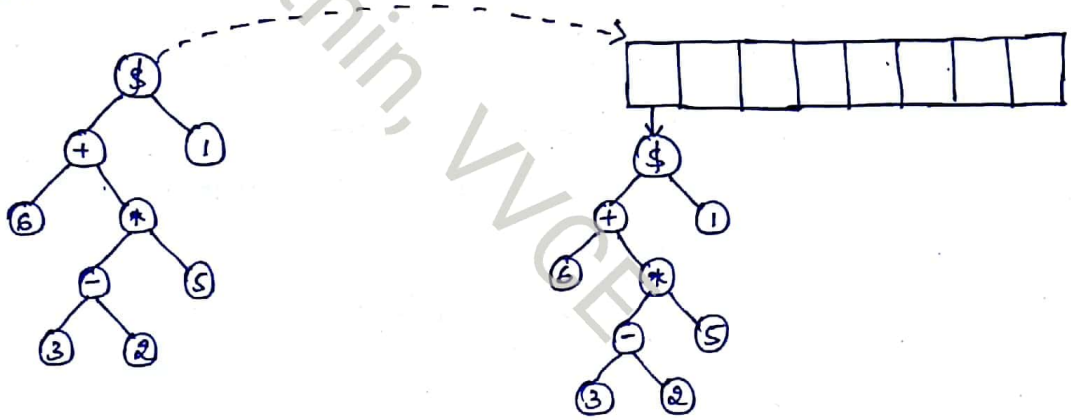
Nithin, VCE

Scanned by CamScanner

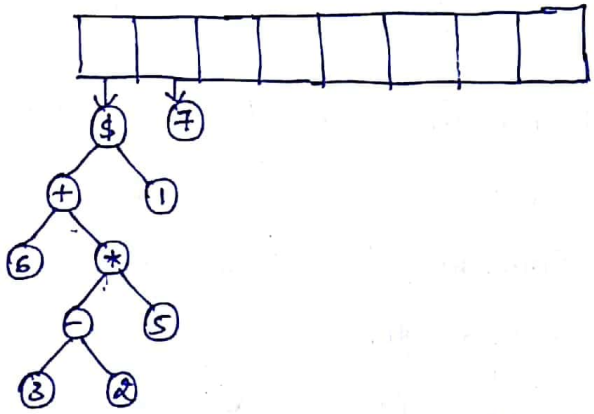
→ Next Symbol is '1' (operand) just push onto STACK



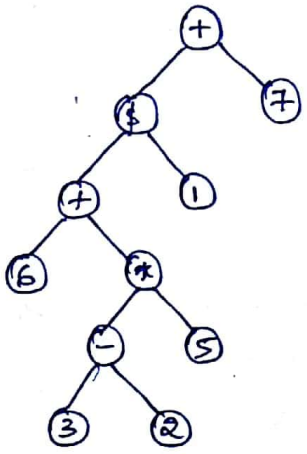
→ The Next Symbol is '$' (operator), pop 2 items & construct them push result.



→ Next Symbol is '7' (operand) just push onto STACK



→ The last Symbol '+' (operator), pop 2 items & construct tree.



Nithin, VVCE

Scanned by CamScanner

→ C Function to Implement Expression Tree

```
Node create_tree (char postfix[])
{
    Node temp, st[20];
    int i, k;
    char Symbol;
    for (i=k=0; (Symbol) = postfix[i] != '\0'; i++)
    {
        temp = getnode();
        temp->info = Symbol;
        temp->rlink = temp->llink = NULL;
        if (isalnum(Symbol))
            st[k++] = temp;          // operand (push)
        else
        {
            temp->rlink = st[--k];   // operator (pop)
            temp->llink = st[--k];
            st[k++] = temp;
        }
    }
    return st[--k];
}
```

* Construct a Binary tree for given Exp

3+4*(7-6)/4+3

Traverse the above generated tree using inorder, preorder & postorder.

⇒ The given Expression is in infix, firstly convert that to postfix (you can convert using general Method / STACK method)

3+4*(7-6)/4+3 — $T_1 = 76-$

3+4*T_1/4+3 — $T_2 = 4T_1*$

3+T_2/4+3 — $T_3 = T_2 4 /$

3+T_3+3 — $T_4 = 3T_3+$

T_4+3

T_4 3+

Nithin, VCE

Scanned by CamScanner

Perform Back Substitution

$= T_4\ 3+$

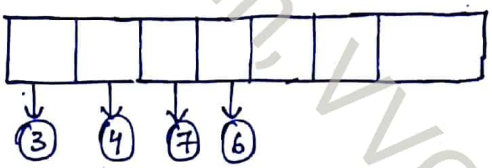
$= 3T_3+3+$

$= 3T_2 4/+3+$

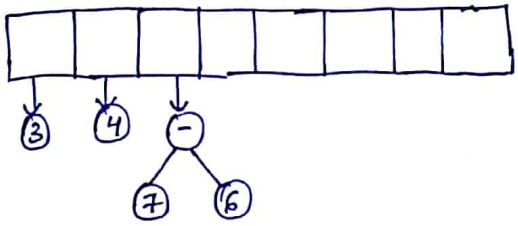
$= 34T_1*4/+3+$

postfix $= 3476-*4/+3+$ //

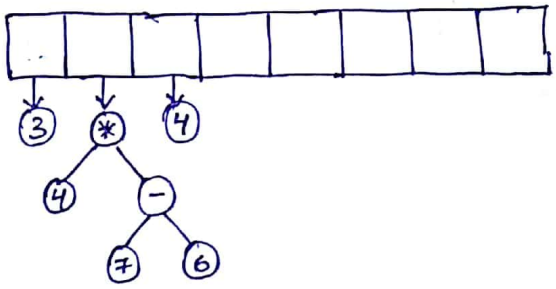
⇒ Expression Tree Construction using postfix.



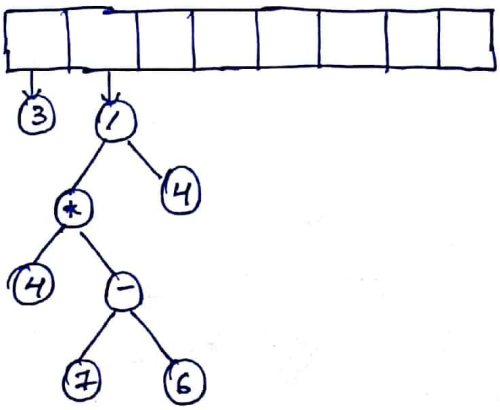
→ Next '−' operator



→ Next '*' operator



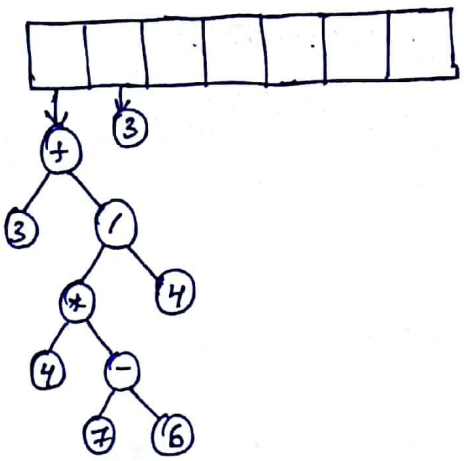
→ Next '/' operator



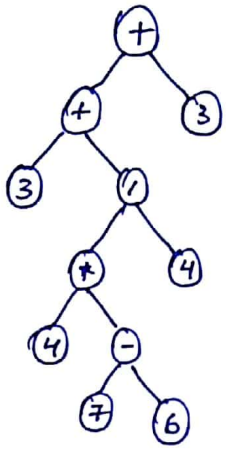
Nithin, VCE

Scanned by CamScanner

→ Next operation '*'

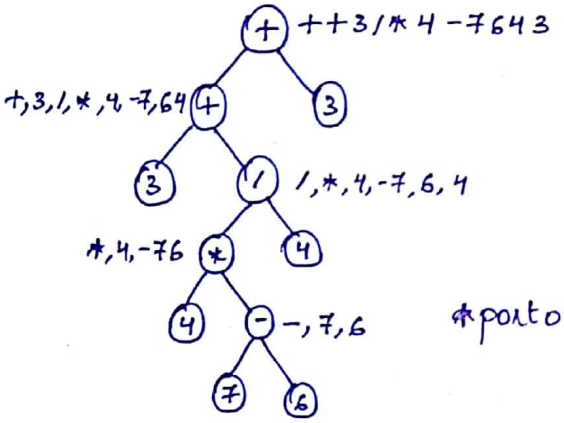


→ The last Symbol is '+' operation

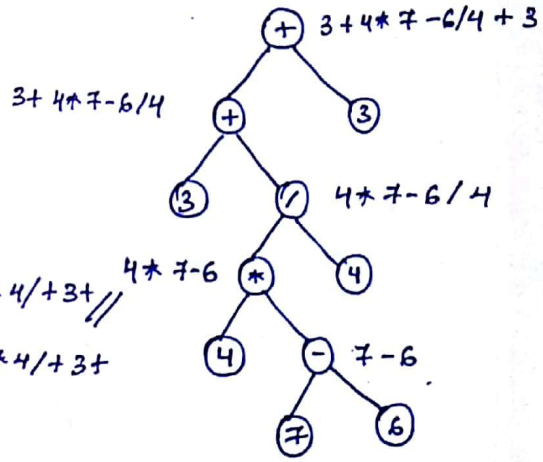


→ Apply preorder, Inorder & postorder Traversal

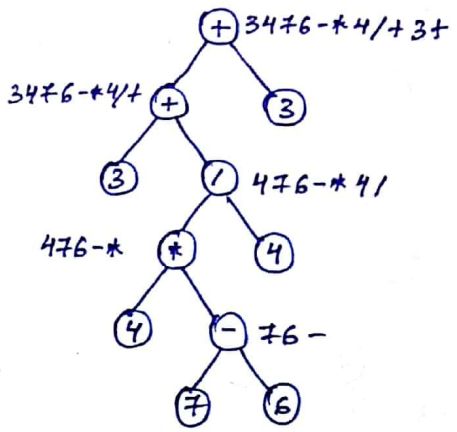
* preorder = ++3/*4-7643 //



* Inorder = 3+4*7-6/4+3 //



* postorder = 3476-*4/+3+ //



Nithin, VVCE

Scanned by CamScanner

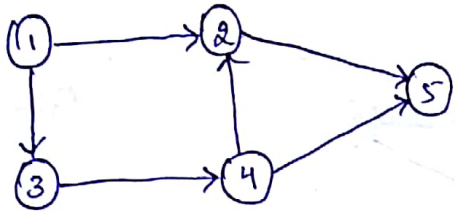
Graphs (Draft Notes)

A Graph 'G' is defined as a pair of two sets V & E denoted by

$G = (V, E)$

where V is Set of vertices & E is set of Edges.

→ Consider the following Ex Graph



where $V = \{1, 2, 3, 4, 5\}$ $|V| = 6$

$E = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 2,5\rangle, \langle 3,4\rangle, \langle 4,2\rangle, \langle 4,5\rangle\}$ $|E| = 6$

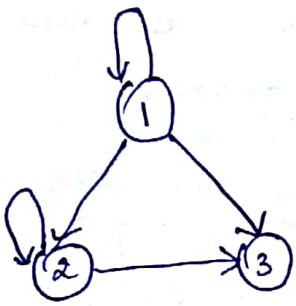
→ A vertex is a Synonym for a node. A vertex is normally represented by a Circle

If 'u' & 'v' are vertices, then an arc or a line joining two vertices 'u' & 'v' is called an Edge.

→ There are two types of Graphs

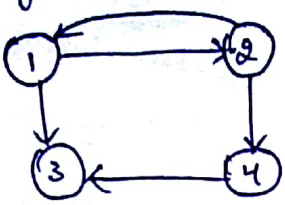
* Directed Graph → G(V, E) in which Every Edge is directed is called directed Graph.
* undirected Graph → G(V, E) in which Every Edge is undirected is called undirected Graph.

→ "Self-loop" is an Edge which Starts & Ends on the Same vertex

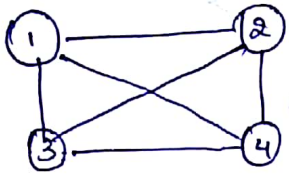


Scanned by CamScanner

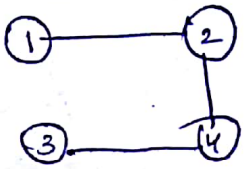
→ "Multigraph" is a Graph with Multiple Occurrence of the same Edge between any two vertices is called Multigraph.



→ "Complete Graph" is a Graph, if there exists an Edge b/w every pair of vertices Such Graph is said to be 'Complete'.



(Connected)

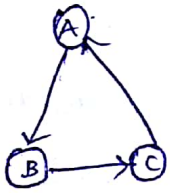


(Disconnected)

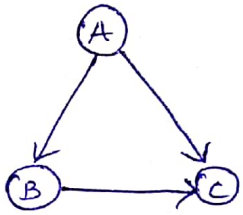
* otherwise, the Graph is said to be "Not complete"

:) The Graphs can be Again classified into two types based on Cycles. In it

* cyclic graph → which contain cycles
* Acyclic graph → which doesn't have cycles.



(cyclic)



{ weighted Graph
{ unweighted Graph

→ "Degree of a Node" is the no of Edges Incident on it.

* In-degree - No of incoming Edges.
* out-degree - No of outgoing Edges

→ "path" is a Sequence of vertices from vertex 'u' to vertex 'v' in a directed/undirected graph.

A path, which starts & Ends on the Same vertex is called "cycle"

"Length of a path" is the no of Edges in the path.

Scanned by CamScanner

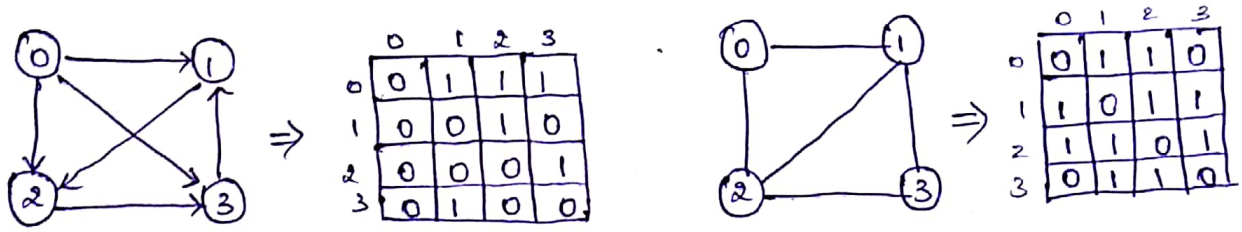
* Representation of Graph :- The Graphs can be represented in two diff ways

  * Adjacency Matrix
  * Adjacency Linked List

* Adjacency Matrix → is a Square Matrix (0,1) used to represent a finite Graph.

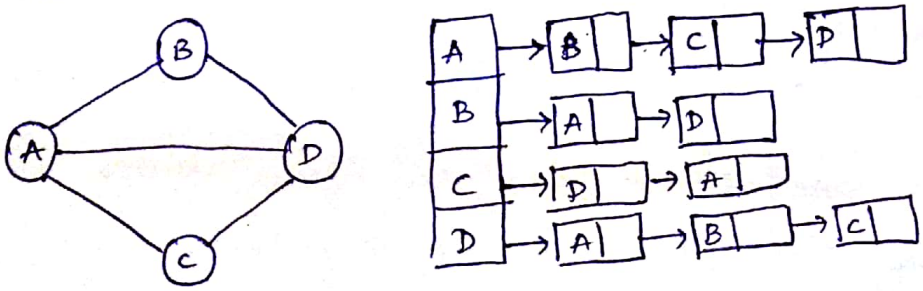
The Elements of the Matrix indicates whether pairs of Vertices are adjacent (reachable) or not in the Graph.

$$A[i][j] = \begin{cases} 1 & \text{If there is Edge from vertex } u \text{ to vertex } v \\ 0 & \text{If there is no Edge from } u \text{ to } v \end{cases}$$



→ Adjacency Matrix repn is Easy to represent & feasible as long as the Graph is Small & connected.
for large Graph, whose Matrix is Sparse, adjacency Matrix repn wastes a lot of Memory.

* Adjacency List representation :- is a way in which graphs can be represented in computer Memory. This Structure consists of list of all nodes in G.
Further, Every node in turn linked to its own list that contains the name of all other nodes that are adjacent (reachable) to it



Scanned by CamScanner

* Advantages of Adjacency list are
  * It is Easy & clear representation of a Graph
  * It is well-suited for Sparse Graph.

* Graph Traversal :- is a technique used for Searching a vertex in a graph, A graph traversal finds the edges to be used in the Search process without creating loops.

→ Graph traversal is applied on vertices of graph without getting into looping path

→ There are two Graph traversal techniques
  * DFS (Depth First Search)
  * BFS (Breadth First Search)

* DFS (Depth first Search) :- is a traversal of a graph, produces a "Spanning tree" as final result. "spanning tree" is a graph without any loops.

→ We use STACK data structure ~~with MAX size of total no~~ to implement DFS traversal of a graph.

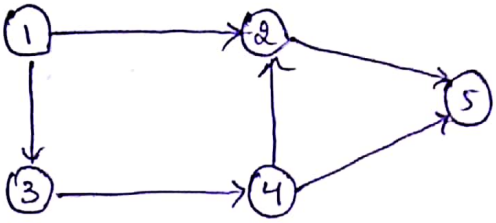
→ Algorithm to implement DFS
* Step 1 :- Define a Stack of size total no of vertices in graph
* Step 2 :- Select any vertex as "Starting vertex" for traversal & visit that vertex & push it on to the Stack.
* Step 3 :- Visit any one of "Adjacent vertex" of the vertex which is at top of stack & not visited & push it on to STACK.
* Step 4 :- Repeat step 3 until there are no new vertex to be visited from the vertex on top of STACK.

Scanned by CamScanner

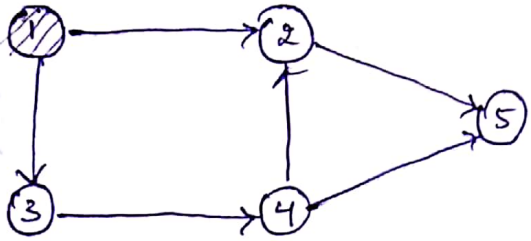
* Step 5: when there is no new vertex to be visited then use "Back Tracking" & pop one vertex from STACK.

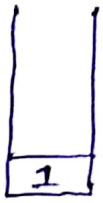
* Step 6: Repeat steps 3, 4, & 5 until STACK becomes Empty.

→ Consider the full Example

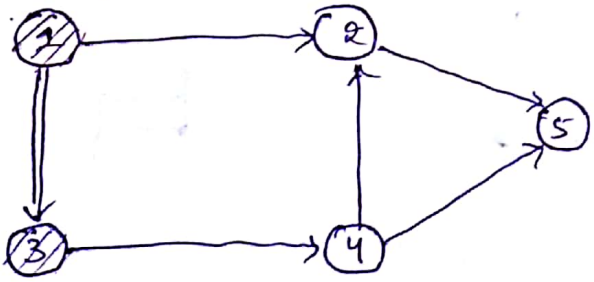


→ Lets start with vertex '1' as Starting vertex



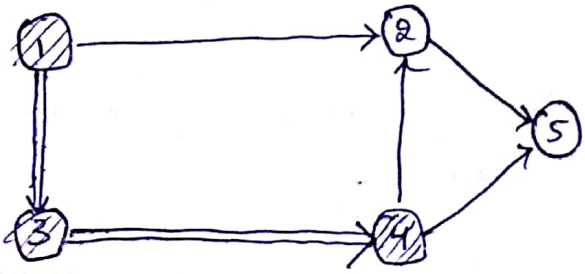


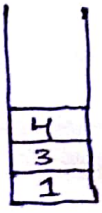
→ presently '1' is visited, Search for Adjacent vertex. i.e we've '2' & '3' Adjacent vertices choose anyone



We've visited '3' so '3' is pushed into STACK

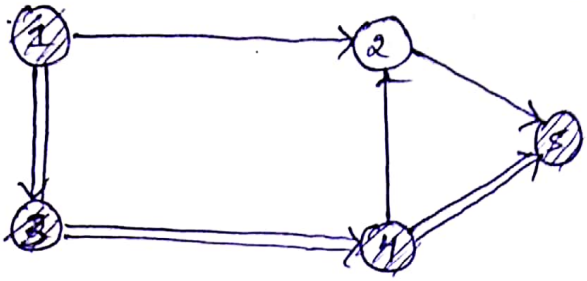
→ from '3' we've only one Adjacent vertex '4', which is not visited, push it on to STACK, Mark it as visited

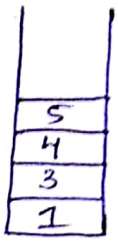




Scanned by CamScanner

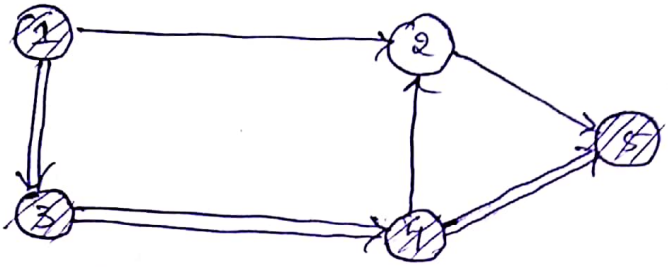
→ From node 4, we've two adjacent vertices '2' & '5', so choose anyone Mark it visited & push to stack. '5' is chosen

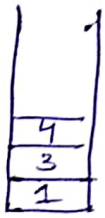




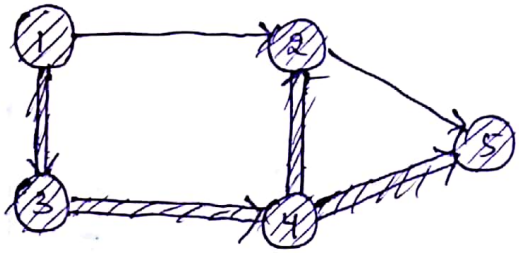
→ There is no adjacent vertex attached to node '5', so we need to perform pop.

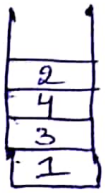
After poping '5', we've '4' one the top of STACK



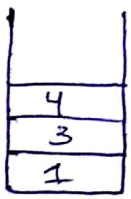


→ Now, the node on top of STACK is '4', which has two adjacent vertices but node 5 is already visited so we're left with only one option '2', Mark it visited & push ~~it~~ to STACK



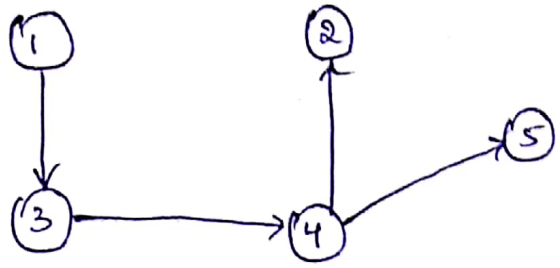


→ presently we've '2' on top of STACK, is there any vertex which is not visited is reachable from '2'. ~~If yes~~ No so we'll pop 2 from STACK



→ presently, we've '4' on top of STACK, is there any vertex which is not visited? "NO" so pop 4 from STACK

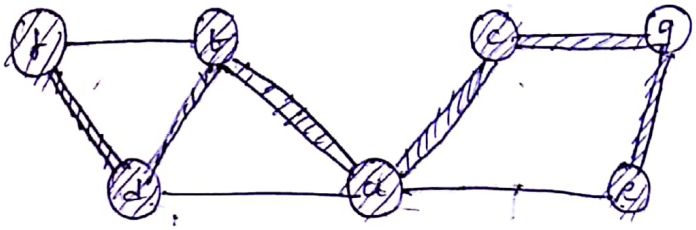
Scanned by CamScanner

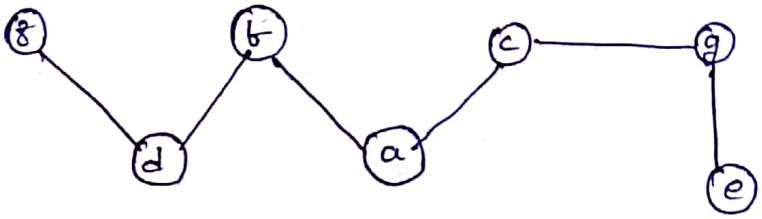


→ The C function to implement DFS

```
void DFS (int u)
{
    int v;
    s[u] = 1;
    print ("%d", u);
    for (v=0; v<n; v++)
    {
        if (a[u][v] == 1 && s[v] == 0)
            dfs(v);
    }
}
```

* DFS:





```
Void DFS (int u)
{
    int v;
    s[u] = 1;
    print ("%d", u);
    for (v=0; v<n; v++)
    {
        if (a[u][v] == 1 && s[v] == 0)
        {
            dfs(v);
```

Scanned by CamScanner

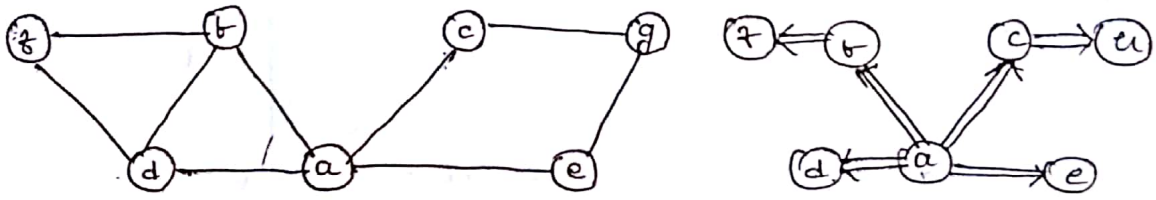
* **BFS:-** The Breadth first Search is a Method of traversing the graph from an arbitary vertex u.
  * first, visit the node u, then we visit all neighbour of u. then we visit the neighbour of neighbour of u & so on.
  * we visit all the neighboring nodes first before moving to next level neighbour.

The Search will terminate when all the vertices have been visited.

→ The BFS Traversal can be Implemented using a "Queue".

→ The Step-wise procedure in BFS
  * Add a src node/vertex from the graph to a Queue of nodes to be visited
  * Visit the topmost node in the Queue & Mark it visited.
  * If that node has any neighbour check to see if they have been visited or not.
  * Add any neighboring node that still need to be visited to the Queue.
  * Remove the node we've visited from the Queue

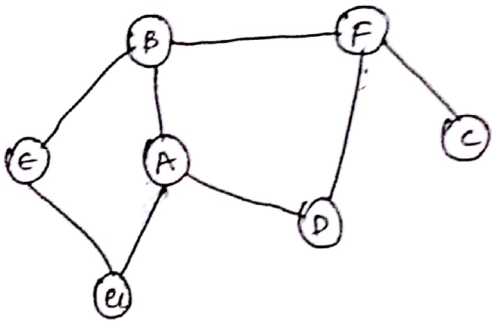
→ Consider the foll Example problem src = a

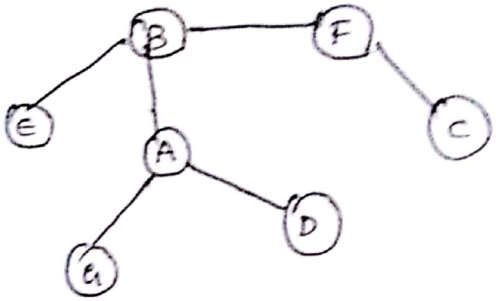


u = del(Q)	v = adj to u	Nodes visited S	Queue
–	–	a	a
a	b c d e	a, b, c, d, e	b, c, d, e
b	a d f	a, b, c, d, e, f	c, d, e, f
c	a, g	a, b, c, d, e, f, g	d, e, f, g
d	a, b, f	a, b, c, d, e, f, g	e, f, g
e	a, g	a, b, c, d, e, f, g	f, g
f	b, d	a, b, c, d, e, f, g	g
g	c, e	a, b, c, d, e, f, g	–

Scanned by CamScanner

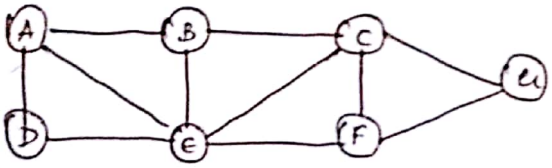
* Consider the foll problem

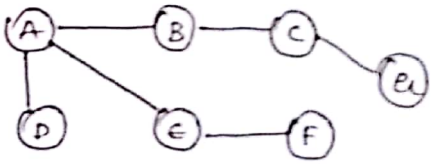




u = del q	v = adj. to u	visited	queue
—	—	A	A
A	B, D, G	A, B, D, G	B, D, G
B	A, E, F	A, B, D, G, E, F	D, G, E, F
D	A, F	A, B, D, G, E, F	G, E, F
G	A, E	A, B, D, G, E, F	E, F
E	B, G	A, B, D, G, E, F	F
F	C, D	A, B, C, D, G, E, F	C
C	F	A, B, C, D, G, E, F	—

* Consider the foll problem src = a





u = del(q)	v = adj to u	visited	queue
—	—	a	a
a	B, D, E	A, B, D, E	B, D, E
B	A, E, C	A, B, D, E, C	D, E, C
D	A, E	A, B, C, D, E	E, C
E	B, C, F	A, B, C, D, E, F	C, F
C	E, F, G	A, B, C, D, E, F, G	F, G
F	C, E, G	A, B, C, D, E, F, G	G
G	C, F	A, B, C, D, E, F, G	—

Scanned by CamScanner

→ C function to implement bfs

```
void bfs (int a[10][10], int u)
{
  int j, r, q[10], v;
  int s[10] = {0};   // initialize s of all element too
  j=0  r=-1;
  q[++r] = u;        // insert u to q
  s[u] = 1;          // Mark as visited
  printf("%d", u);   // print visited.
while (j<=r)
  {
    u = q[j++];      // delete u from q
    for (v=0; v<n; v++)
      {
        if (a[u][v]==1)   // check for edge
          {
            if (s[v]==0)  // not visited
              {
                printf("%d", v); // visited
                s[v] = 1;   // Mark it visited
                q[++r] = v; // insert to q
              }
          }
      }
}
```

* difference b/w DFS & BFS

DFS	BFS
* Tests all possibilities in one path until it needs to backtrack	* Visits all nearest neighbours & then their neighbours
* DFS goes to bottom of subtree then backtracks	* BFS finds shortest path to destn.
* Runs till no unvisited vertex left	—||—
* Implemented using STACK	* Implemented using QUEUE
* Consumes less Memory	* requires more Memory

Scanned by CamScanner

* Data Hierarchy :- refers to the systematic org of data, often in hierarchical form.

→ The data hierarchy involves various data items such as

* data field → is an elementary unit of info that stores a single fact or an attribute
  Ex: Name, USN, ph no etc

* Record → is a collection of related data fields.
  Ex. Student record contain name, USN, Mark etc

* File → A file is a collection of related records
  Ex: Student Academic info, Student personal info etc

* Database → is a data structure that stores organized info usually in the form of tabular columns.
A student file consisting of "Student name", "phone", "USN" etc can be represented in the form of table.

* File Attributes :- File attributes are settings that are associated with computer files. These attributes may grant or deny certain rights to the user or OS when a file is accessed.

→ The various file attributes supported by Window OS from Microsoft are

* Read-only → If a file is marked read-only, it cannot be modified or deleted

* Hidden → If a file is marked hidden, when we see the contents of directory, we cannot see the file.

* System → A file marked as a s/m file indicates that it is used by computer s/m. not be modified / deleted

* Directory → when a directory bit is set, the name associated with file is considered as directory

* Archive → bit is set, if is ready for backup, After backup is over, archive bit is cleared.

Scanned by CamScanner

* Text & Binary file:

* Text file → Standard text document that contains only the Made up of letters 0-9, A-Z & special symbols. Each character in file is stored using ASCII code.

* Binary file → Stored in binary form 0's & 1's. We cannot read & understand binary file. All Executable files are binary files

* Basic file operation → Creation
Updation
Retrieval
Maintaince.

* File org:

→ Sequential file org

→ Relative file org.

→ Indexed sequential file → Seq + Relative

R_0	R_1 --- R_n

R_0
R_1
.
R_3
R_5

Index	File
1	R_1
2	R_2
3	R_3
4	R_4
5	
6	

```
Void bfs (int a[10][10], int u)
{ int d, r, v, q[10];
  int s[10] = {0};
  d=0; r=-1;
  s[u] = 1;
  q[++r] = u;
  printf("%d", u);
  while (d <= r)
  {
    u = q[d++];
    for(v=0; v<n; v++)
    { if (a[u][v] == 1)
      { if (s[v] == 0)
        {
          q[++r] = v;
          s[v] = 1;
          printf("%d", v);
        }
      }
    }
  }
```

Scanned by CamScanner

* Hashing :- It is a process of Transforming a String of characters into a usually shorter fixed length value or key that represents the original String.

→ Hashing is used to index & retrieve items in a database because it is faster to find the item using the shorter hashed key than to find it using original value.
The hashing technique is Essentially Independent of 'n' where 'n' is no of Elements.

* Hash function :- It is any function that can be used to map data of arbitary Size to data of a fixed Size.
The values returned by a hash function are called hash values, hash code, digests or Simply hashes.

* Hash table :- data can be Stored in the form of a table using arrays with the help of hash function which gives a hash value as an index to access any Element in the table.

→ There are two types of hashing techniques
  * Static hashing → table whose Size is fixed.
  * dynamic hashing → table that grows & shrinks.

→ Consider the foll Ex of creating hash table for Elements 11, 12, 13, 14, 15

* total no of Elements are '5' so, we need hash table with 5 slots

A[0]	A[1]	A[2]	A[3]	A[4]
15	11	12	13	14

* Now, we need to choose hash function

$H(k) = k \% 5$. k = key value

$H(11) = 11 \% 5 = 1$ $H(14) = 14 \% 5 = 4$
$H(12) = 12 \% 5 = 2$ $H(15) = 15 \% 5 = 0$
$H(13) = 13 \% 5 = 3$

Scanned by CamScanner

→ The table obtained after inserting each of item using hash value as index

A[0]	A[1]	A[2]	A[3]	A[4]
15	11	12	13	14

table created using elements & hash value is the hash table.

→ Consider a hash table ht for storing various identifiers.

⇒ we need to store identifiers in a hash table. let us assume all identifiers starts with only letters 'A' to 'Z' having range 0 to 25.

→ ~~bat~~ there are 26 letters, let us have a hash table ht whose size is $m = 26$.

Assume identifiers to be inserted are : Ant, dog, cat, bat, Eagle, fish & Ape.

* Hash value of 'Ant' = 'A' starts with 'A' = $0 \% 26 = 0$
* Hash value of 'dog' = starts with 'D' = $3 \% 26 = 3$
* Hash value of 'cat' = starts with 'C' = $2 \% 26 = 2$

bat = $1 \% 26 = 1$

Eagle = $4 \% 26 = 4$

fish = $5 \% 26 = 5$

ape = $0 \% 26 = 0$

ht[]	Identifiers
0	Ant
1	bat
2	cat
3	dog
4	eagle
5	fish
⋮	⋮
25	

"Ape" cannot be inserted ht[0] because, an identifier is already placed. This condition is called "Overflow or Collision".

Scanned by CamScanner

* Collision (Overflow handling) If the size of hash table is fixed then two or more keys will have the same hash value. The phenomenon of two or more keys being hashed to the same location of hash table is called "collision".

→ There are two important collision resolving strategies
  * open Addressing (Linear probing)
  * Chaining.

* ~~Hash~~ Different types of hash function :- A good hash function should satisfy all criteria.
  * Hash value is fully determined by the data being hashed.
  * "uniformly" distributes the data across the entire set of possible hash values
  * Computation should be simple.

→ The various types of hash functions are
  * Division Method
  * Mid-Square Method
  * folding Method.

* Division Method :- It is the most simplest method of hashing an integer x.
  This method divides x by M then uses the remainder obtained.

$$h(x) = x \% M$$

This division method is quite good for just about any value of M & since it requires only a single division operation.

→ The major problem of the division Method is that while using this Method, Consecutive keys Map to Consecutive hash values.

→ Ex $h(123) = 123 \% 100 = 23$ //

Scanned by CamScanner

* Mid-Square Method :- In this Method, the key k is Squared. A number in the middle of k^2 is Selected by removing the digits from both ends.

$$h(k) = l$$

where 'l' is obtained by removing digits from both ends of k^2.

→ for Ex: $k = 2345$ its Square $k^2 = 5748\,25$

$h(2345) = 45$

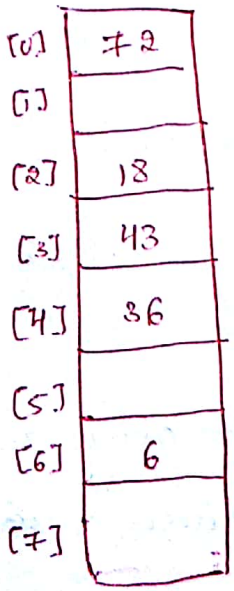
by discarding 2 digit in begining & 2 digits from the end.

* Folding Method :- In this Method, the key k is divided into no of parts $k_1, k_2, k_3 \dots k_n$ of same length except the last part. Then all parts are added together to get Hash value.

$$h(k) = k_1 + k_2 + k_3 \dots k_n$$

→ for Ex. $k = 123987234876$, partitions are $k_1 = 12$ $k_2 = 39$ $k_3 = 87$ $k_4 = 23$ $k_5 = 48$. $k_6 = 76$

$$h(k) = 12 + 39 + 87 + 23 + 48 + 76 = 285 //$$



Add key 10, 5 & 15

Scanned by CamScanner

Course Title: Course Code:

Question Number	Solution	Marks Allocated

* Linear probing / open Addressing :- There are 3 steps

  * Initialize hash table
  * Select Hash function
  * find the Index of Each location

* Initialize Hash table:- Assume the size of hash table is 5

a[0]	a[1]	a[2]	a[3]	a[4]
0	0	0	0	0

```
for (i=0; i< Hash_Size; i++)
    {
    a[i] = 0;
    }
```

* Select Hash function :- Division Method

$$h(k) = k \% \text{Hash_table_size}$$

```
int H (int k)
 {
    return k % HASH_Size ;
 }
```

* finding the Index of Each location (insertion)

```
void Insert hash_table (int Item, int a[])
 {
    int i, Index, h_value)
     h_value = H (Item);
     for (i=0; i< Hash_Size ) i++)
     {
      Index = (hash_value + i) % Hash_Size;
          if (a[Index] == 0)
                break;
     }
    if (a[Index] == 0)
          a[Index] = Item;
    Else
       print ("Hash table is full\n");
 }
```

Scanned by CamScanner

* Insertion Sort :-

The Insertion Sort is similar to the way we play cards. After shuffling the cards, we pick each card & insert it into the proper place so that cards in hand are ordered.

→ The Insertion-Sort is an in-place comparison based Sorting Algorithm.
Here, a sub-list is maintained which is always sorted.

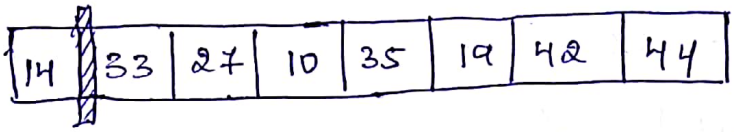
→ The working procedure of Insertion Sort-

Step 1 :- the lower part of an array is maintained to be sorted

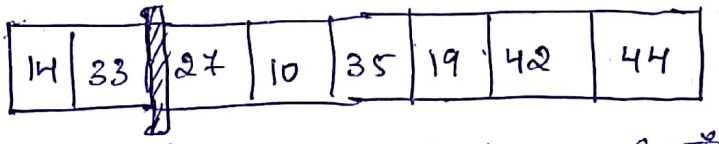
Step 2 :- choose the element to be inserted sorted-list

Step 3 :- find it Appropriate place & then it has to be inserted

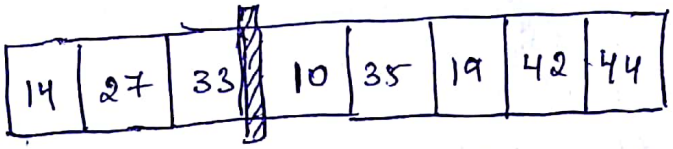
→ Consider the foll Example



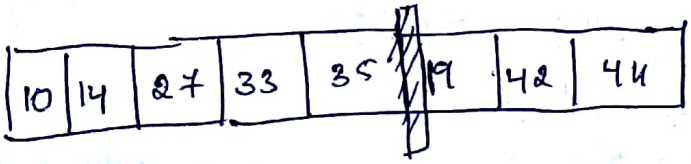
Compare A[i] = Item = 33 with 'j' i.e j = i-1, in the sorted order.

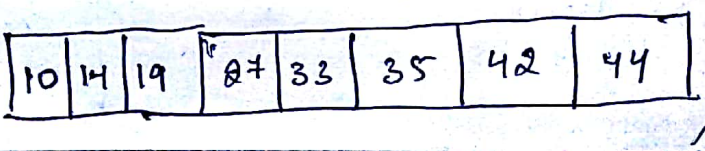


compare A[2] = 27 with sorted Array [illegible], [illegible], so it has to be inserted b/w 14 & 33



compare A[3] = 10 with sorted Array





//

Scanned by CamScanner

→ Algorithm for Insertion Sort

* Step 1: If it is first element, it is already sorted, pick the next element

```
for (i = 1; i < n; i++)
    item = a[i];
```

* Step 2: Compare with all elements in Sorted Sublist

```
j = i - 1;
```

* Step 3: Shift all the elements in the sorted sublist that is greater than the item

```
while (item < a[j] && j >= 0)
{
    a[j+1] = j;
    j = j - 1;
}
```

* Step 4: Insert the item

```
a[j+1] = item;
```

→ Function to perform Insertion Sort

```
void insertion sort (int a[], int n)
{
    int i, j, item;
    for (i = 1; i < n; i++)
    {
        item = a[i];
        j = i - 1;
        while (item < a[j] && j >= 0)
        {
            a[j+1] = a[j];
            j = j - 1;
        }
        a[j+1] = item;
    }
}
```

* Advantage
  - ⇒ Simplest Sorting Algo

* disadvantage
  - → great inefficiency for large Arrays
  - → Not as quick as Merge sort or quicksort.

Scanned by CamScanner